

✓ Stock Market Prediction from News Headlines

Comparative Study of NLP Techniques for DJIA Movement Prediction

Course: INM434 NLP Coursework

Student: Sneha Naidu (210048420)

Date: April 2026

Objective

Compare multiple NLP approaches to predict stock market movement from news headlines:

- **BoW + Logistic Regression** (baseline)
- **TF-IDF + Logistic Regression** (improved)
- **TF-IDF + SVM** (classical ML)
- **DistilBERT** (transformer-based)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import pickle
from sklearn.feature_extraction.text import CountVectorizer, TfidfVec
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_s
from collections import Counter
import warnings
warnings.filterwarnings('ignore')

# try:
#     from wordcloud import WordCloud
#     print("✓ WordCloud imported")
# except ImportError:
#     print("⚠ WordCloud not found – install with: pip install word
```

```
# =====
# Setup: Define Paths
# =====
DATA_PATH = 'data/raw/Combined_News_DJIA.csv'
```

```

MODELS_DIR = 'models/'
OUTPUTS_DIR = 'outputs/'
GRAPHS_DIR = 'Graphs/'

# Create directories if they don't exist
import os
os.makedirs(MODELS_DIR, exist_ok=True)
os.makedirs(OUTPUTS_DIR, exist_ok=True)
os.makedirs(GRAPHS_DIR, exist_ok=True)

print(f"✓ Data path: {DATA_PATH}")
print(f"✓ Models directory: {MODELS_DIR}")
print(f"✓ Outputs directory: {OUTPUTS_DIR}")
print(f"✓ Graphs directory: {GRAPHS_DIR}")

```

```

✓ Data path: data/raw/Combined_News_DJIA.csv
✓ Models directory: models/
✓ Outputs directory: outputs/
✓ Graphs directory: Graphs/

```

✓ Step 1: Load Dataset

```

# =====
# Step 1: Load and Explore the Dataset
# =====
df = pd.read_csv(DATA_PATH)

# CSV columns: Date(0) | Label(1) | Top1-Top25(2-26)
label = df.iloc[:, 1].values # Extract Label column (column index 1)

# Display basic info
print(f"Dataset shape: {df.shape}")
print(f"Columns: {list(df.columns[:5])}... (27 total)")
print(f"\nLabel distribution (after loading):")
print(f"  Class 0 (DOWN): {(label == 0).sum()}")
print(f"  Class 1 (UP): {(label == 1).sum()}")
print(f"\nFirst 3 rows info:")
print(df.head(3))

```

```

Dataset shape: (1989, 27)
Columns: ['Date', 'Label', 'Top1', 'Top2', 'Top3']... (27 total)

```

```

Label distribution (after loading):
  Class 0 (DOWN): 924
  Class 1 (UP): 1065

```

```

First 3 rows info:

```

	Date	Label	Top1	Top2	Top3
0	2008-08-08	0	b'Georgia 'downs two Russian warplanes' as cou.		
1	2008-08-11	1	b'Why wont America and Nato help us? If they w.		
2	2008-08-12	0	b'Remember that adorable 9-year-old who sang a.		
				b'BREAKING: Musharraf to be impeached.'	

```

1 b'Bush puts foot down on Georgian conflict'
2 b''Russia 'ends Georgia operation'''

Top3 \
0 b'Russia Today: Columns of troops roll into So...
1 b''Jewish Georgian minister: Thanks to Israeli ...
2 b''If we had no sexual harassment we would hav...

Top4 \
0 b'Russian tanks are moving towards the capital...
1 b'Georgian army flees in disarray as Russians ...
2 b'Al-Qa'eda is losing support in Iraq because ...

Top5 \
0 b'Afghan children raped with 'impunity,' U.N. ...
1 b'Olympic opening ceremony fireworks 'faked'''
2 b'Ceasefire in Georgia: Putin Outmaneuvers the...

Top6 \
0 b'150 Russian tanks have entered South Ossetia...
1 b'What were the Mossad with fraudulent New Zea...
2 b'Why Microsoft and Intel tried to kill the X0...

Top7 \
0 b''Breaking: Georgia invades South Ossetia, Rus...
1 b'Russia angered by Israeli military sale to G...
2 b'Stratfor: The Russo-Georgian War and the Bal...

Top8 ... \
0 b''The 'enemy combatent' trials are nothing but...
1 b'An American citizen living in S.Ossetia blam...
2 b'I'm Trying to Get a Sense of This Whole Geor...

Top16 \
0 b'Georgia Invades South Ossetia - if Russia ge...
1 b'Israel and the US behind the Georgian aggres...
2 b'U.S. troops still in Georgia (did you know t...

Top17 \
0 b'Al-Qaeda Faces Islamist Backlash'
1 b''Do not believe TV, neither Russian nor Geor...
2 b'Why Russias response to Georgia was right'
```

✓ Step 2: Preprocessing - Combine & Clean Headlines

```

def combine_headlines(data):
    """
    Combine Top1-Top25 headlines into single text document
    Remove non-alphabetic characters and lowercase
    """
    headlines = []
    for row in range(len(data)):
        # Combine columns 2-27 (headlines)
        text = ' '.join(str(x) for x in data.iloc[row, 1:26])
        # Remove non-alphabetic characters
        text = re.sub("[^a-zA-Z]", " ", text)
```

```

# Lowercase
text = text.lower()
# Remove extra spaces
text = ' '.join(text.split())
headlines.append(text)
return headlines

# Apply preprocessing
print("Processing headlines...")
all_headlines = combine_headlines(df)

print(f"✓ {len(all_headlines)} documents created")
print(f"\nSample processed headline:")
print(f"Length: {len(all_headlines[0])} chars")
print(f"Text: {all_headlines[0][:200]}...")

# Derived text-length features used later in EDA
word_counts = [len(text.split()) for text in all_headlines]
char_counts = [len(text) for text in all_headlines]

```

Processing headlines...
✓ 1989 documents created

Sample processed headline:
Length: 2219 chars
Text: b georgia downs two russian warplanes as countries move to brink

✓ Step 2.5: Exploratory Data Analysis & Visualization

```

# =====
# Step 2.5(a): Class Distribution Analysis
# =====
# Create analysis DataFrame with proper label column
analysis_df = pd.DataFrame({
    'headline': all_headlines,
    'word_count': word_counts,
    'char_count': char_counts,
    'label': label # Use the label variable extracted from CSV column
})

print(f"Analysis DataFrame shape: {analysis_df.shape}")
print(f"Label distribution:")
print(analysis_df['label'].value_counts().sort_index())

# =====
# Step 2.5(b): Class Distribution Visualization
# =====
# Extract counts
count_down = int((analysis_df['label'] == 0).sum())
count_up = int((analysis_df['label'] == 1).sum())
total = count_down + count_up

```

```

# Calculate percentages
percentage_down = (count_down / total * 100) if total > 0 else 0
percentage_up = (count_up / total * 100) if total > 0 else 0

print(f"\nClass distribution:")
print(f"  DOWN (0): {count_down} headlines ({percentage_down:.1f}%)"
print(f"  UP (1): {count_up} headlines ({percentage_up:.1f}%)"

# Create visualization
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# --- Subplot 1: Bar Chart ---
x_pos = [0, 1]
counts = [count_down, count_up]
colors_class = ['#e74c3c', '#2ecc71'] # Red for DOWN, Green for UP

bars = axes[0].bar(x_pos, counts, color=colors_class, edgecolor='black')
axes[0].set_ylabel('Number of Headlines', fontsize=11, fontweight='bold')
axes[0].set_title('Class Distribution - Bar Chart', fontsize=12, fontweight='bold')
axes[0].set_xticks(x_pos)
axes[0].set_xticklabels(['DOWN (0)', 'UP (1)'], fontsize=10)
axes[0].grid(axis='y', alpha=0.3, linestyle='--')

# Add value labels on bars
for i, (bar, count) in enumerate(zip(bars, counts)):
    height = bar.get_height()
    axes[0].text(bar.get_x() + bar.get_width()/2., height,
                  f'{int(count)}\n({percentage_down if i == 0 else percentage_up:.1f}%',
                  ha='center', va='bottom', fontsize=10, fontweight='bold')

# --- Subplot 2: Pie Chart ---
if count_down > 0 and count_up > 0:
    labels_text = [f'DOWN (0)\n{count_down}\n({percentage_down:.1f}%',
                   f'UP (1)\n{count_up}\n({percentage_up:.1f}%',
    axes[1].pie([count_down, count_up], labels=labels_text, colors=colors_class,
                autopct='%1.1f%%', startangle=90, textprops={'fontsize': 10, 'fontweight': 'bold'},
                wedgeprops={'edgecolor': 'black', 'linewidth': 1.5})
    axes[1].set_title('Class Distribution - Pie Chart', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.savefig(f'{GRAPHS_DIR}class_distribution.png', dpi=300, bbox_inches='tight')
plt.show()

print(f"\nClass distribution plot saved.")
print(f"Balance: {percentage_down:.1f}% DOWN vs {percentage_up:.1f}% UP")

```

Analysis DataFrame shape: (1989, 4)

Label distribution:

label

0 924

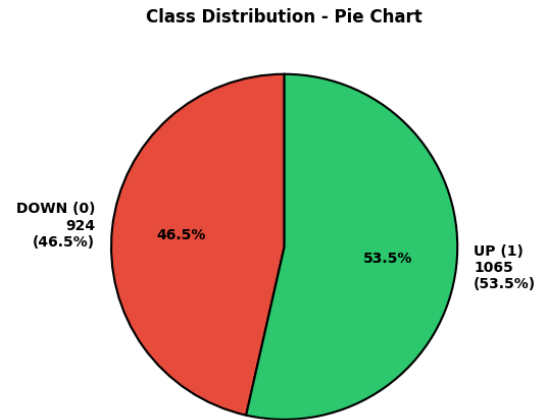
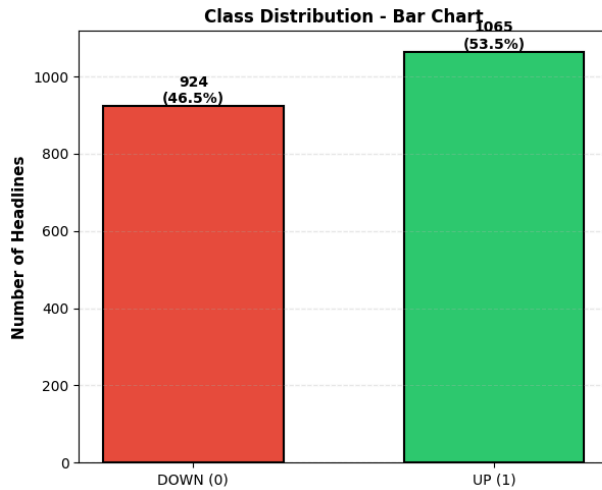
1 1065

Name: count, dtype: int64

Class distribution:

DOWN (0): 924 headlines (46.5%)

UP (1): 1065 headlines (53.5%)



Class distribution plot saved.

Balance: 46.5% DOWN vs 53.5% UP

```
# =====
# DIAGNOSTIC: Check Data Integrity
# =====
print("\n" + "=" * 80)
print("🔍 DATA INTEGRITY CHECK:")
print("=" * 80)
print(f"\nOriginal CSV (df):")
print(f"  df.shape: {df.shape}")
print(f"  df rows: {len(df)}")

print(f"\nProcessed headlines (all_headlines):")
print(f"  len(all_headlines): {len(all_headlines)}")

print(f"\nAnalysis DataFrame:")
print(f"  analysis_df.shape: {analysis_df.shape}")
```

```

print(f"  len(analysis_df): {len(analysis_df)}")

print(f"\nFirst 2 samples:")
for i in range(min(2, len(analysis_df))):
    print(f"    [{i}] Label={analysis_df.iloc[i]['label']}, Words={anal
    print(f"        Text: {analysis_df.iloc[i]['headline'][:80]}...")

if len(all_headlines) != len(df):
    print(f"\n⚠️  ISSUE DETECTED:")
    print(f"    all_headlines length ({len(all_headlines)}) != df leng
    print(f"    → The combine_headlines() function may have failed!")

```

🔍 DATA INTEGRITY CHECK:

Original CSV (df):
 df.shape: (1989, 27)
 df rows: 1989

Processed headlines (all_headlines):
 len(all_headlines): 1989

Analysis DataFrame:
 analysis_df.shape: (1989, 4)
 len(analysis_df): 1989

First 2 samples:

```

[0] Label=0, Words=393
    Text: b georgia downs two russian warplanes as countries move t
[1] Label=1, Words=285
    Text: b why wont america and nato help us if they wont help us

```

```

# =====
# Step 2.5(c): Text Length Distribution Analysis
# =====
# Split data by class
headlines_down = [all_headlines[i] for i in range(len(all_headlines))
headlines_up = [all_headlines[i] for i in range(len(all_headlines)) i
word_counts_down = [word_counts[i] for i in range(len(word_counts)) i
word_counts_up = [word_counts[i] for i in range(len(word_counts)) if

print(f"Text length summary:")
print(f"  DOWN: min={min(word_counts_down)}, max={max(word_counts_dow
print(f"  UP:   min={min(word_counts_up)}, max={max(word_counts_up)},

# =====
# Visualization: Histograms + Box Plot
# =====
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# --- Subplot 1: Overall Distribution Histogram ---
axes[0].hist([word_counts_down, word_counts_up], bins=50, label=['DOW

```

```
        color=['#e74c3c', '#2ecc71'], edgecolor='black', alpha=0.7)
axes[0].set_xlabel('Word Count', fontsize=11, fontweight='bold')
axes[0].set_ylabel('Frequency', fontsize=11, fontweight='bold')
axes[0].set_title('Text Length Distribution (Overall)', fontsize=12,
axes[0].legend(fontsize=10)
axes[0].grid(axis='y', alpha=0.3)

# --- Subplot 2: Per-Class Histograms ---
axes[1].hist(word_counts_down, bins=40, label='DOWN (0)', color='#e74c3c',
edgecolor='black', alpha=0.7)
axes[1].hist(word_counts_up, bins=40, label='UP (1)', color='#2ecc71',
edgecolor='black', alpha=0.7)
axes[1].set_xlabel('Word Count', fontsize=11, fontweight='bold')
axes[1].set_ylabel('Frequency', fontsize=11, fontweight='bold')
axes[1].set_title('Text Length Distribution (By Class)', fontsize=12,
axes[1].legend(fontsize=10)
axes[1].grid(axis='y', alpha=0.3)

# --- Subplot 3: Box Plot ---
bp = axes[2].boxplot([word_counts_down, word_counts_up], labels=['DOWN (0)', 'UP (1)'],
patch_artist=True, widths=0.6)
for patch, color in zip(bp['boxes'], ['#e74c3c', '#2ecc71']):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)
for whisker in bp['whiskers']:
    whisker.set(linewidth=1.5)
axes[2].set_ylabel('Word Count', fontsize=11, fontweight='bold')
axes[2].set_title('Word Count Distribution (Box Plot)', fontsize=12,
axes[2].grid(axis='y', alpha=0.3)

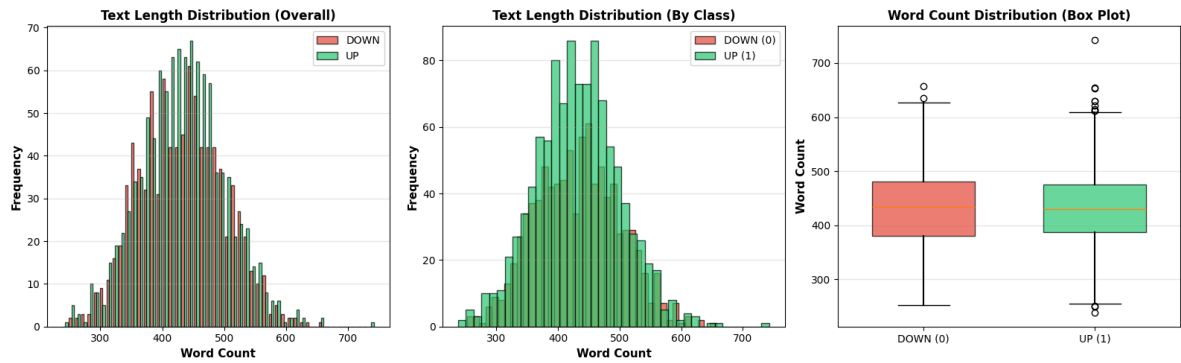
plt.tight_layout()
plt.savefig(f'{GRAPHS_DIR}text_length_distribution.png', dpi=300, bbox_inches='tight')
plt.show()

print(f"\nText length plots saved.")
```


Text length summary:

DOWN: min=252, max=657, mean=432.4

UP: min=238, max=743, mean=432.0



Text length plots saved.

```
# =====
# Step 2.5(d): Top 20 Most Frequent Words by Class
# =====
from collections import Counter

# Combine headlines for each class
text_down = ' '.join([h if isinstance(h, str) else str(h) for i, h in
text_up = ' '.join([h if isinstance(h, str) else str(h) for i, h in e

# Split into words and get top 20
words_down = Counter(text_down.split()).most_common(20)
words_up = Counter(text_up.split()).most_common(20)

print(f"Top word frequencies by class:")
print(f"\nMost frequent words in DOWN class:")
for i, (word, count) in enumerate(words_down[:5], 1):
    print(f" {i}. '{word}': {count} occurrences")

print(f"\nMost frequent words in UP class:")
for i, (word, count) in enumerate(words_up[:5], 1):
    print(f" {i}. '{word}': {count} occurrences")

# =====
```

```
# Visualization: Horizontal Bar Charts
# =====:
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# --- LEFT: Top 20 words for DOWN ---
words_down_list, counts_down_list = zip(*words_down)
axes[0].barh(range(len(words_down_list)), counts_down_list, color='#e
axes[0].set_yticks(range(len(words_down_list)))
axes[0].set_yticklabels(words_down_list, fontsize=9)
axes[0].set_xlabel('Frequency', fontsize=11, fontweight='bold')
axes[0].set_title('Top 20 Words - DOWN (Class 0)', fontsize=12, fontw
axes[0].invert_yaxis()
axes[0].grid(axis='x', alpha=0.3)

# --- RIGHT: Top 20 words for UP ---
words_up_list, counts_up_list = zip(*words_up)
axes[1].barh(range(len(words_up_list)), counts_up_list, color='#2ecc7
axes[1].set_yticks(range(len(words_up_list)))
axes[1].set_yticklabels(words_up_list, fontsize=9)
axes[1].set_xlabel('Frequency', fontsize=11, fontweight='bold')
axes[1].set_title('Top 20 Words - UP (Class 1)', fontsize=12, fontwei
axes[1].invert_yaxis()
axes[1].grid(axis='x', alpha=0.3)

plt.tight_layout()
plt.savefig(f'{GRAPHS_DIR}top20_words_comparison.png', dpi=300, bbox_
plt.show()

print(f"\nTop word comparison plot saved.")
```

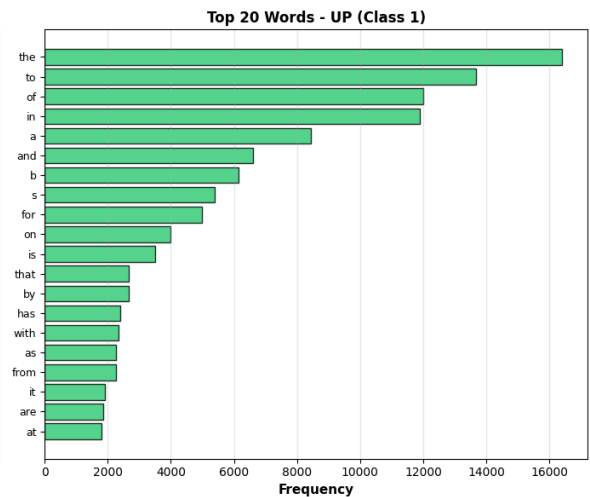
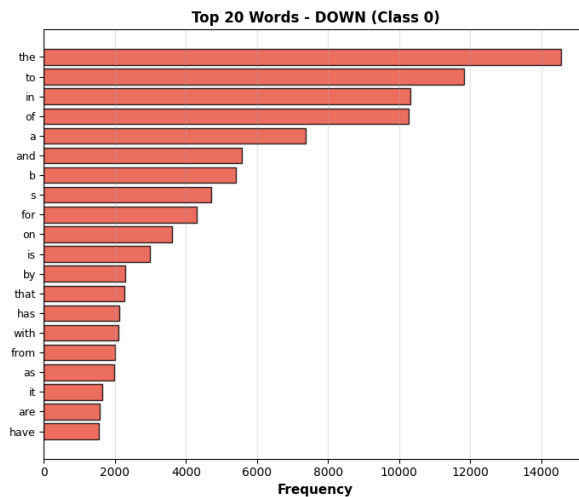
Top word frequencies by class:

Most frequent words in DOWN class:

1. 'the': 14554 occurrences
2. 'to': 11823 occurrences
3. 'in': 10306 occurrences
4. 'of': 10278 occurrences
5. 'a': 7371 occurrences

Most frequent words in UP class:

1. 'the': 16391 occurrences
2. 'to': 13667 occurrences
3. 'of': 12011 occurrences
4. 'in': 11901 occurrences
5. 'a': 8445 occurrences



Top word comparison plot saved.

```
# =====
# Step 2.5(e): Word Clouds Visualization
# =====
try:
    from wordcloud import WordCloud
```

```
# Create figure for dual word clouds
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# --- LEFT: Word Cloud for DOWN class ---
wc_down = WordCloud(width=600, height=400, background_color='white',
                    colormap='Reds', max_words=100).generate(text_)
axes[0].imshow(wc_down, interpolation='bilinear')
axes[0].set_title('Word Cloud - DOWN (Class 0)', fontsize=12, fontw
axes[0].axis('off')

# --- RIGHT: Word Cloud for UP class ---
wc_up = WordCloud(width=600, height=400, background_color='white',
                 colormap='Greens', max_words=100).generate(text_)
axes[1].imshow(wc_up, interpolation='bilinear')
axes[1].set_title('Word Cloud - UP (Class 1)', fontsize=12, fontw
axes[1].axis('off')

plt.tight_layout()
plt.savefig(f'{GRAPHS_DIR}wordclouds_up_down.png', dpi=300, bbox_
plt.show()

print(f"Word clouds generated and saved.")

except ImportError:
    print("wordcloud library not installed. Install with: pip install
```



Step 3: Train/Test Split (Time-based)

```
# First check if Date column exists
print(f"Columns: {df.columns.tolist()}")

# Time-based split (financial data respects temporal order)
# Split at 2015 boundary
split_date = '20150101'

train_mask = df.iloc[:, 0] < split_date # Assuming first column is Date
test_mask = df.iloc[:, 0] >= split_date

train_indices = np.where(train_mask)[0]
test_indices = np.where(test_mask)[0]

# Create train/test sets
X_train = [all_headlines[i] for i in train_indices]
X_test = [all_headlines[i] for i in test_indices]

y_train = label[train_indices]
y_test = label[test_indices]

print(f"Training set: {len(X_train)} samples")
print(f"  Class 0 (Down): {(y_train == 0).sum()}")
print(f"  Class 1 (Up): {(y_train == 1).sum()}")
print(f"\nTest set: {len(X_test)} samples")
print(f"  Class 0 (Down): {(y_test == 0).sum()}")
print(f"  Class 1 (Up): {(y_test == 1).sum()}")

# Store results dictionary
results = {}
```

```
Columns: ['Date', 'Label', 'Top1', 'Top2', 'Top3', 'Top4', 'Top5', 'Top6', 'Top7', 'Top8', 'Top9', 'Top10']
Training set: 1863 samples
  Class 0 (Down): 868
  Class 1 (Up): 995

Test set: 126 samples
  Class 0 (Down): 56
  Class 1 (Up): 70
```

Step 3.5: Methodology

Design Choices

1. Time-Based Split (NOT random split)

- Financial data is **temporal** — future prices depend on past events
- Random split would cause **data leakage** (training on future news to predict past movements)

- Proper approach: Split chronologically at 2015 → Train on 2012-2014, test on 2015+
- This reflects real-world prediction scenario (predict tomorrow based on past patterns)

2. TF-IDF vs Bag of Words

- **BoW**: Simple word frequency → treats "stock rising" and "stock falling" equally
- **TF-IDF**: Weights rare/informative terms → "bankruptcy" signals DOWN, "profit" signals UP
- TF-IDF captures **semantic importance** → Financial terms are discriminative
- Trade-off: Slightly more complex but captures financial language patterns

3. Why Embeddings/Transformers Matter

- **Classical (BoW, TF-IDF)**: No context → "bank" (financial vs riverbank) is ambiguous
- **BERT embeddings**: Contextual → "bank fails" vs "river bank" have different representations
- **Why BERT > other embeddings**: Pre-trained on 100M+ documents → learns financial semantics
- BERT captures phrase-level patterns: "stock rally" is different from "rally stocks"

Step 4: Model 1 - Bag of Words + Logistic Regression (Baseline)

```
# Model 1: BoW + Logistic Regression
print("=" * 60)
print("Model 1: Bag of Words + Logistic Regression")
print("=" * 60)

# Vectorization
bow = CountVectorizer(ngram_range=(1,1), max_features=5000)
X_train_bow = bow.fit_transform(X_train)
X_test_bow = bow.transform(X_test)

print(f"Vocabulary size: {len(bow.get_feature_names_out())}")
print(f"Train shape: {X_train_bow.shape}")
print(f"Test shape: {X_test_bow.shape}")

# Train model
model_bow_lr = LogisticRegression(max_iter=1000, random_state=42, cla
model_bow_lr.fit(X_train_bow, y_train)

# Predictions
y_pred_bow_lr = model_bow_lr.predict(X_test_bow)
cm_bow_lr = confusion_matrix(y_test, y_pred_bow_lr)
```

```
# Evaluation
acc_bow_lr = accuracy_score(y_test, y_pred_bow_lr)
prec_bow_lr = precision_score(y_test, y_pred_bow_lr)
rec_bow_lr = recall_score(y_test, y_pred_bow_lr)
f1_bow_lr = f1_score(y_test, y_pred_bow_lr)

print(f"\nResults:")
print(f"  Accuracy: {acc_bow_lr:.4f}")
print(f"  Precision: {prec_bow_lr:.4f}")
print(f"  Recall: {rec_bow_lr:.4f}")
print(f"  F1-Score: {f1_bow_lr:.4f}")

# Store results
results['BoW + LogReg'] = {
    'accuracy': acc_bow_lr,
    'precision': prec_bow_lr,
    'recall': rec_bow_lr,
    'f1': f1_bow_lr,
    'cm': cm_bow_lr,
    'model': model_bow_lr,
    'vectorizer': bow
}

# Feature importance
feature_names = bow.get_feature_names_out()
coefficients = model_bow_lr.coef_[0]
top_words_idx = np.argsort(np.abs(coefficients))[-10:]
print(f"\nTop features:")
for idx in reversed(top_words_idx):
    print(f"  {feature_names[idx]}: {coefficients[idx]:.4f}")

# Confusion Matrix Visualization
fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(cm_bow_lr, annot=True, fmt='d', cmap='Blues', cbar=True,
            xticklabels=['DOWN (0)', 'UP (1)'],
            yticklabels=['DOWN (0)', 'UP (1)'],
            annot_kws={'size': 14, 'weight': 'bold'},
            linewidths=2, linecolor='black')
ax.set_title('Model 1: BoW + Logistic Regression\nConfusion Matrix',
            fontsize=13, fontweight='bold', pad=15)
ax.set_ylabel('True Label', fontsize=11, fontweight='bold')
ax.set_xlabel('Predicted Label', fontsize=11, fontweight='bold')
plt.tight_layout()
plt.savefig(f'{GRAPHS_DIR}cm_model1_bow_logreg.png', dpi=300, bbox_inches='tight')
plt.show()
```

Model 1: Bag of Words + Logistic Regression

Vocabulary size: 5000
 Train shape: (1863, 5000)
 Test shape: (126, 5000)

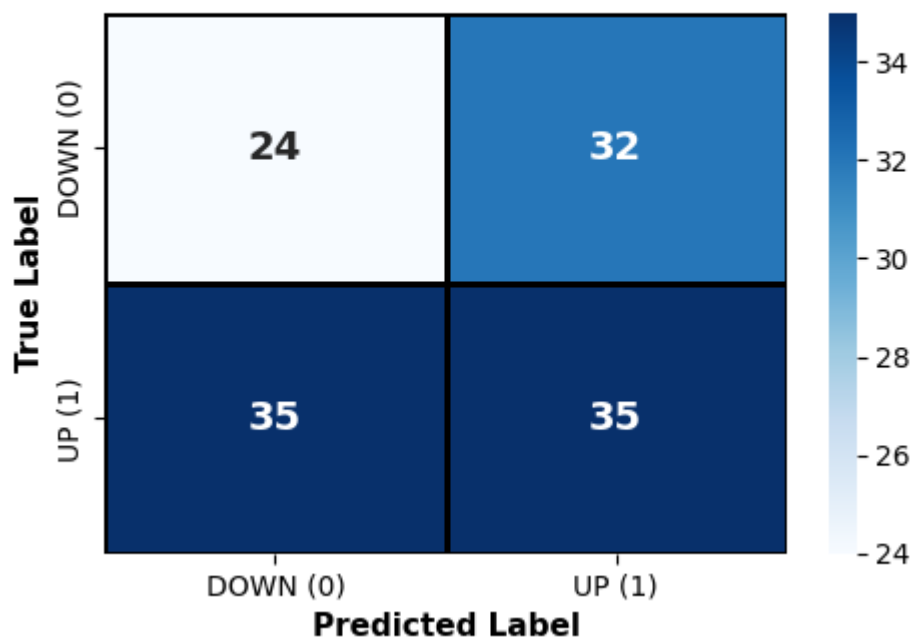
Results:

Accuracy: 0.4683
 Precision: 0.5224
 Recall: 0.5000
 F1-Score: 0.5109

Top features:

low: -0.7621
 territory: 0.7617
 begin: -0.7498
 sanctions: -0.7015
 did: -0.6831
 mali: 0.6605
 comments: 0.6517
 joint: 0.6435
 congo: -0.6385
 tv: 0.6341

Model 1: BoW + Logistic Regression Confusion Matrix



✓ Step 5: Model 2 - TF-IDF + Logistic Regression

```
# Model 2: TF-IDF + Logistic Regression
print("\n" + "=" * 60)
print("Model 2: TF-IDF + Logistic Regression")
print("=" * 60)

# Vectorization (main model uses unigrams + bigrams)
```



```

tfidf = TfidfVectorizer(ngram_range=(1,2), max_features=5000)
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print(f"TF-IDF vocabulary size: {len(tfidf.get_feature_names_out())}")
print(f"Train shape: {X_train_tfidf.shape}")
print(f"Test shape: {X_test_tfidf.shape}")

# Train model
model_tfidf_lr = LogisticRegression(max_iter=1000, random_state=42, c
model_tfidf_lr.fit(X_train_tfidf, y_train)

# Predictions
y_pred_tfidf_lr = model_tfidf_lr.predict(X_test_tfidf)
cm_tfidf_lr = confusion_matrix(y_test, y_pred_tfidf_lr)

# Evaluation
acc_tfidf_lr = accuracy_score(y_test, y_pred_tfidf_lr)
prec_tfidf_lr = precision_score(y_test, y_pred_tfidf_lr)
rec_tfidf_lr = recall_score(y_test, y_pred_tfidf_lr)
f1_tfidf_lr = f1_score(y_test, y_pred_tfidf_lr)

print(f"\nResults:")
print(f"  Accuracy: {acc_tfidf_lr:.4f}")
print(f"  Precision: {prec_tfidf_lr:.4f}")
print(f"  Recall: {rec_tfidf_lr:.4f}")
print(f"  F1-Score: {f1_tfidf_lr:.4f}")

# Store results
results['TF-IDF + LogReg'] = {
    'accuracy': acc_tfidf_lr,
    'precision': prec_tfidf_lr,
    'recall': rec_tfidf_lr,
    'f1': f1_tfidf_lr,
    'cm': cm_tfidf_lr,
    'model': model_tfidf_lr,
    'vectorizer': tfidf
}

# Feature importance
feature_names_tfidf = tfidf.get_feature_names_out()
coefficients_tfidf = model_tfidf_lr.coef_[0]
top_words_tfidf_idx = np.argsort(np.abs(coefficients_tfidf))[-10:]
print(f"\nTop features:")
for idx in reversed(top_words_tfidf_idx):
    print(f"  {feature_names_tfidf[idx]}: {coefficients_tfidf[idx]:.4

# Confusion Matrix Visualization
fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(cm_tfidf_lr, annot=True, fmt='d', cmap='Greens', cbar=True,
            xticklabels=['DOWN (0)', 'UP (1)'],
            yticklabels=['DOWN (0)', 'UP (1)'],
            annot_kws={'size': 14, 'weight': 'bold'},
            linewidths=2, linecolor='black')
ax.set_title('Model 2: TF-IDF + Logistic Regression\nConfusion Matrix

```

```

        fontsize=13, fontweight='bold', pad=15)
ax.set_ylabel('True Label', fontsize=11, fontweight='bold')
ax.set_xlabel('Predicted Label', fontsize=11, fontweight='bold')
plt.tight_layout()
plt.savefig(f'{GRAPHS_DIR}cm_model2_tfidf_logreg.png', dpi=300, bbox_
plt.show()
print("\nTF-IDF confusion matrix saved.")

# -----
# Small but high-impact preprocessing experiments
# -----
def evaluate_tfidf_variant(name, vectorizer):
    X_train_var = vectorizer.fit_transform(X_train)
    X_test_var = vectorizer.transform(X_test)

    model_var = LogisticRegression(max_iter=1000, random_state=42, cl
    model_var.fit(X_train_var, y_train)

    y_pred_var = model_var.predict(X_test_var)
    return {
        'accuracy': accuracy_score(y_test, y_pred_var),
        'precision': precision_score(y_test, y_pred_var),
        'recall': recall_score(y_test, y_pred_var),
        'f1': f1_score(y_test, y_pred_var)
    }

ngram_unigram_metrics = evaluate_tfidf_variant(
    'Unigram TF-IDF',
    TfidfVectorizer(ngram_range=(1,1), max_features=5000)
)

ngram_bigram_metrics = evaluate_tfidf_variant(
    'Bigram TF-IDF',
    TfidfVectorizer(ngram_range=(1,2), max_features=5000)
)

stopword_metrics = evaluate_tfidf_variant(
    'TF-IDF + English Stopword Removal',
    TfidfVectorizer(ngram_range=(1,2), max_features=5000, stop_words=
)

tfidf_experiment_summary = pd.DataFrame({
    'Unigram TF-IDF (1,1)': ngram_unigram_metrics,
    'Bigram TF-IDF (1,2)': ngram_bigram_metrics,
    'Bigram TF-IDF + stop_words=english': stopword_metrics
}).T

print("\n" + "=" * 60)
print("TF-IDF preprocessing experiments")
print("=" * 60)
print(tfidf_experiment_summary[['accuracy', 'precision', 'recall', 'f

ngram_f1_gain = ngram_bigram_metrics['f1'] - ngram_unigram_metrics['f
stopword_f1_delta = stopword_metrics['f1'] - ngram_bigram_metrics['f1

```

```

if ngram_f1_gain >= 0:
    print(f"\nAdding bigrams improved F1 by {ngram_f1_gain:.4f}.")
else:
    print(f"\nAdding bigrams reduced F1 by {abs(ngram_f1_gain):.4f} i

if stopword_f1_delta >= 0:
    print(f"Stopword removal improved F1 by {stopword_f1_delta:.4f}."
else:
    print(f"Stopword removal reduced F1 by {abs(stopword_f1_delta):.4

# -----
# Time-aware cross-validation
# -----
# Use chronological splits on the training portion only to avoid look
cv_splitter = TimeSeriesSplit(n_splits=5)
cv_results = {}

cv_pipelines = {
    'BoW + LogReg': Pipeline([
        ('vectorizer', CountVectorizer(ngram_range=(1,1), max_feature
        ('model', LogisticRegression(max_iter=1000, random_state=42,
    ]),
    'TF-IDF + LogReg': Pipeline([
        ('vectorizer', TfidfVectorizer(ngram_range=(1,2), max_feature
        ('model', LogisticRegression(max_iter=1000, random_state=42,
    ]),
    'TF-IDF + SVM': Pipeline([
        ('vectorizer', TfidfVectorizer(ngram_range=(1,2), max_feature
        ('model', SVC(kernel='linear', random_state=42, class_weight=
    ]),
    'TF-IDF + NB': Pipeline([
        ('vectorizer', TfidfVectorizer(ngram_range=(1,2), max_feature
        ('model', MultinomialNB())
    ])
}

print("\n" + "=" * 60)
print("Time-series cross-validation (training set only)")
print("=" * 60)
for model_name, pipeline in cv_pipelines.items():
    cv_scores = cross_val_score(pipeline, X_train, y_train, cv=cv_spl
    cv_results[model_name] = {
        'cv_f1_mean': float(cv_scores.mean()),
        'cv_f1_std': float(cv_scores.std())
    }
    print(f"{model_name}: F1 = {cv_scores.mean():.4f} ± {cv_scores.st

```



```
=====
Model 2: TF-IDF + Logistic Regression
=====
TF-IDF vocabulary size: 5000
Train shape: (1863, 5000)
Test shape: (126, 5000)
```

Results:

Accuracy: 0.4921

Recall: 0.5143

F1-Score: 0.5294

Step 6: Model 3 - TF-IDF + SVM

```
# Model 3: TF-IDF + SVM
print("\n" + "=" * 60)
print("MODEL 3: TF-IDF + SUPPORT VECTOR MACHINE")
print("=" * 60)

# Reuse the best TF-IDF preprocessing found in Model 2 if available
if 'tfidf_experiment_summary' in locals():
    best_tfidf_variant = tfidf_experiment_summary['f1'].idxmax()
    print(f"Using best TF-IDF variant for SVM: {best_tfidf_variant}")
    if best_tfidf_variant == 'Unigram TF-IDF (1,1)':
        tfidf_svm = TfidfVectorizer(ngram_range=(1,1), max_features=5000)
    elif best_tfidf_variant == 'Bigram TF-IDF (1,2)':
        tfidf_svm = TfidfVectorizer(ngram_range=(1,2), max_features=5000)
    else:
        tfidf_svm = TfidfVectorizer(ngram_range=(1,2), max_features=5000)
    X_train_tfidf_svm = tfidf_svm.fit_transform(X_train)
    X_test_tfidf_svm = tfidf_svm.transform(X_test)
else:
    print("tfidf_experiment_summary not found; using the current bigram")
    tfidf_svm = tfidf
    X_train_tfidf_svm = X_train_tfidf
    X_test_tfidf_svm = X_test_tfidf

# Train model
model_tfidf_svm = SVC(kernel='linear', random_state=42, verbose=0, class_weight='balanced')
print("Training SVM (this may take a moment)...")
model_tfidf_svm.fit(X_train_tfidf_svm, y_train)

# Predictions
y_pred_tfidf_svm = model_tfidf_svm.predict(X_test_tfidf_svm)
cm_tfidf_svm = confusion_matrix(y_test, y_pred_tfidf_svm)

# Evaluation
acc_tfidf_svm = accuracy_score(y_test, y_pred_tfidf_svm)
prec_tfidf_svm = precision_score(y_test, y_pred_tfidf_svm)
rec_tfidf_svm = recall_score(y_test, y_pred_tfidf_svm)
f1_tfidf_svm = f1_score(y_test, y_pred_tfidf_svm)

print(f"\nResults:")
print(f"  Accuracy: {acc_tfidf_svm:.4f}")
print(f"  Precision: {prec_tfidf_svm:.4f}")
print(f"  Recall: {rec_tfidf_svm:.4f}")
```

```
print(f"  F1-Score: {f1_tfidf_svm:.4f}")

# Model 3 Confusion Matrix Visualization
fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(cm_tfidf_svm, annot=True, fmt='d', cmap='Oranges', cbar=True,
            xticklabels=['DOWN (0)', 'UP (1)'],
            yticklabels=['DOWN (0)', 'UP (1)'],
            annot_kws={'size': 14, 'weight': 'bold'},
            linewidths=2, linecolor='black')
ax.set_title('Model 3: TF-IDF + Support Vector Machine\nConfusion Mat
            fontsize=13, fontweight='bold', pad=15)
ax.set_ylabel('True Label', fontsize=11, fontweight='bold')
ax.set_xlabel('Predicted Label', fontsize=11, fontweight='bold')
plt.tight_layout()
plt.savefig(f'{GRAPHS_DIR}cm_model3_tfidf_svm.png', dpi=300, bbox_inches='tight')
plt.show()
print("\nSVM confusion matrix saved.")

# Feature importance
feature_names_svm = tfidf_svm.get_feature_names_out()
coefficients_svm = model_tfidf_svm.coef_.toarray()[0]
top_words_svm_idx = np.argsort(np.abs(coefficients_svm))[-10:]

print(f"\nTop 10 important TF-IDF terms (SVM):")
for idx in reversed(top_words_svm_idx):
    print(f"  {feature_names_svm[idx]}: {coefficients_svm[idx]:.4f}")

# Store results
results['TF-IDF + SVM'] = {
    'accuracy': acc_tfidf_svm,
    'precision': prec_tfidf_svm,
    'recall': rec_tfidf_svm,
    'f1': f1_tfidf_svm,
    'cm': cm_tfidf_svm,
    'model': model_tfidf_svm,
    'vectorizer': tfidf_svm
}
```

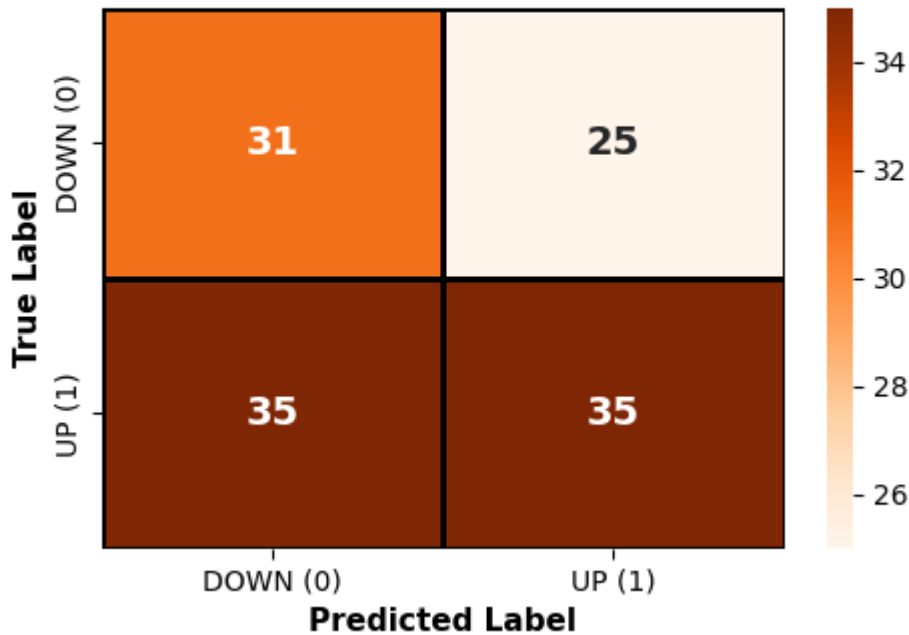
```
=====
MODEL 3: TF-IDF + SUPPORT VECTOR MACHINE
=====
```

Training SVM (this may take a moment)...

Results:

Accuracy: 0.5238
 Precision: 0.5833
 Recall: 0.5000
 F1-Score: 0.5385

MODEL 3: TF-IDF + Support Vector Machine Confusion Matrix



✅ Model 3 Confusion Matrix visualization saved!

Top 10 important TF-IDF terms (SVM):

and: 1.8704
 country: -1.7060
 did: -1.6558
 run: -1.6185
 three: 1.5865
 hacking: -1.5549
 nigeria: 1.5492
 mumbai: 1.5441
 people: -1.4524
 sanctions: -1.4455

✓ Step 7: Model 4 - TF-IDF + Multinomial Naive Bayes

```
# Step 7: Model 4 - TF-IDF + Multinomial Naive Bayes
print("\n" + "=" * 60)
print("Model 4: TF-IDF + Multinomial Naive Bayes")
print("=" * 60)
```

```

# Use the same best TF-IDF configuration as SVM for a fair comparison
tfidf_nb = tfidf_svm
X_train_tfidf_nb = X_train_tfidf_svm
X_test_tfidf_nb = X_test_tfidf_svm

model_tfidf_nb = MultinomialNB()
model_tfidf_nb.fit(X_train_tfidf_nb, y_train)
y_pred_tfidf_nb = model_tfidf_nb.predict(X_test_tfidf_nb)
cm_tfidf_nb = confusion_matrix(y_test, y_pred_tfidf_nb)

acc_tfidf_nb = accuracy_score(y_test, y_pred_tfidf_nb)
prec_tfidf_nb = precision_score(y_test, y_pred_tfidf_nb)
rec_tfidf_nb = recall_score(y_test, y_pred_tfidf_nb)
f1_tfidf_nb = f1_score(y_test, y_pred_tfidf_nb)

print(f"\nResults:")
print(f"  Accuracy: {acc_tfidf_nb:.4f}")
print(f"  Precision: {prec_tfidf_nb:.4f}")
print(f"  Recall: {rec_tfidf_nb:.4f}")
print(f"  F1-Score: {f1_tfidf_nb:.4f}")

# Top features for Naive Bayes based on class log-probabilities
feature_names_nb = tfidf_nb.get_feature_names_out()
class_log_probs = model_tfidf_nb.feature_log_prob_
top_up_nb_idx = np.argsort(class_log_probs[1])[-10:]
top_down_nb_idx = np.argsort(class_log_probs[0])[-10:]

print(f"\nTop 10 TF-IDF terms for UP in Naive Bayes:")
for idx in reversed(top_up_nb_idx):
    print(f"  {feature_names_nb[idx]}: {class_log_probs[1, idx]:.4f}")

print(f"\nTop 10 TF-IDF terms for DOWN in Naive Bayes:")
for idx in reversed(top_down_nb_idx):
    print(f"  {feature_names_nb[idx]}: {class_log_probs[0, idx]:.4f}")

fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(cm_tfidf_nb, annot=True, fmt='d', cmap='Purples', cbar=True,
            xticklabels=['DOWN (0)', 'UP (1)'],
            yticklabels=['DOWN (0)', 'UP (1)'],
            annot_kws={'size': 14, 'weight': 'bold'},
            linewidths=2, linecolor='black')
ax.set_title('Model 4: TF-IDF + Multinomial Naive Bayes\nConfusion Matrix')
ax.set_ylabel('True Label', fontsize=11, fontweight='bold')
ax.set_xlabel('Predicted Label', fontsize=11, fontweight='bold')
plt.tight_layout()
plt.savefig(f'{GRAPHS_DIR}cm_model4_tfidf_nb.png', dpi=300, bbox_inches='tight')
plt.show()
print("\nNB confusion matrix saved.")

# Store results
results['TF-IDF + NB'] = {
    'accuracy': acc_tfidf_nb,
    'precision': prec_tfidf_nb,
    'recall': rec_tfidf_nb,

```



```
'f1': f1_tfidf_nb,  
'cm': cm_tfidf_nb,  
'model': model_tfidf_nb,  
'vectorizer': tfidf_nb,  
'top_up_features': [(feature_names_nb[idx], float(class_log_probs  
'top_down_features': [(feature_names_nb[idx], float(class_log_pro  
}
```

Model 4: TF-IDF + Multinomial Naive Bayes

Results:

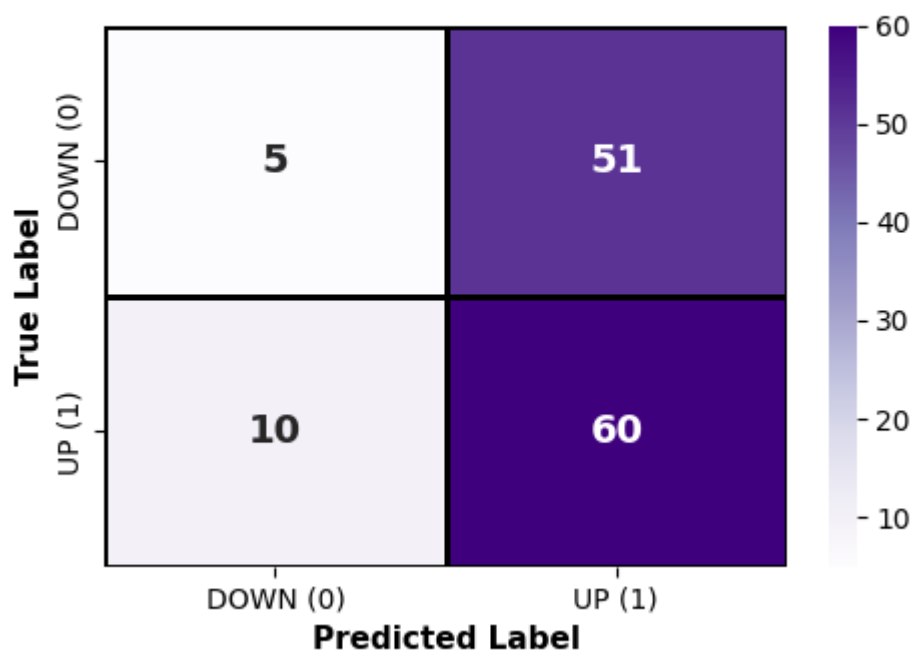
Accuracy: 0.5159
Precision: 0.5405
Recall: 0.8571
F1-Score: 0.6630

Top 10 TF-IDF terms for UP in Naive Bayes:

the: -4.5915
to: -4.7810
of: -4.9093
in: -4.9166
and: -5.4937
for: -5.7584
on: -5.9682
is: -6.0566
that: -6.2898
by: -6.3002

Top 10 TF-IDF terms for DOWN in Naive Bayes:

the: -4.6109
to: -4.8164
in: -4.9554
of: -4.9591
and: -5.5648
for: -5.7953
on: -5.9505
is: -6.1114
by: -6.3520
that: -6.3575

**Model 4: TF-IDF + Multinomial Naive Bayes
Confusion Matrix**

NB confusion matrix saved.

✓ Step 8: Model 5 - FinBERT (Transformer)

```
# Model 5: Fine-tuned FinBERT (transformer with probabilities, no ext
print("\n" + "=" * 60)
print("MODEL 5: FINE-TUNED FINBERT (NO BLACK-BOX API)")
print("=" * 60)

try:
    import inspect
    import io
    import contextlib
    import torch
    import torch.nn as nn
    from datasets import Dataset
    from sklearn.utils.class_weight import compute_class_weight
    from transformers import AutoTokenizer, AutoModelForSequenceClass
    from transformers.utils import logging as hf_logging

    model_name = "ProsusAI/finbert"

    # Step 1: Build HuggingFace datasets from our train/test split
    print("Step 1: Preparing dataset...")
    train_dataset = Dataset.from_dict({
        "text": X_train,
        "labels": y_train.tolist()
    })
    test_dataset = Dataset.from_dict({
        "text": X_test,
        "labels": y_test.tolist()
    })

    # Step 2: Tokenize
    print("Step 2: Tokenizing data...")
    tokenizer = AutoTokenizer.from_pretrained(model_name)

    def tokenize_function(examples):
        return tokenizer(examples["text"], padding="max_length", trun

    train_tokenized = train_dataset.map(tokenize_function, batched=True)
    test_tokenized = test_dataset.map(tokenize_function, batched=True)
    train_tokenized.set_format(type="torch", columns=["input_ids", "at
    test_tokenized.set_format(type="torch", columns=["input_ids", "at

    # Step 3: Load model
    print("Step 3: Loading FinBERT model...")
    config = AutoConfig.from_pretrained(model_name)
    config.num_labels = 2
    config.id2label = {0: "DOWN", 1: "UP"}
    config.label2id = {"DOWN": 0, "UP": 1}

    # Silence mismatch/load reports in notebook output.
    hf_logging.set_verbosity_error()
    with contextlib.redirect_stdout(io.StringIO()), contextlib.redire
```

```

        model = AutoModelForSequenceClassification.from_pretrained(
            model_name,
            config=config,
            ignore_mismatched_sizes=True
        )

# Step 4: Training arguments (compatible across transformers versions)
print("Step 4: Setting up training...")
ta_params = inspect.signature(TrainingArguments.__init__).parameters
training_kwargs = {
    "output_dir": "/tmp/finbert_training",
    "num_train_epochs": 3,
    "per_device_train_batch_size": 16,
    "per_device_eval_batch_size": 16,
    "learning_rate": 2e-5,
    "weight_decay": 0.01,
    "logging_steps": 100
}

optional_kwargs = {
    "overwrite_output_dir": True,
    "save_strategy": "epoch",
    "load_best_model_at_end": True,
    "no_cuda": not torch.cuda.is_available()
}

if "evaluation_strategy" in ta_params:
    optional_kwargs["evaluation_strategy"] = "epoch"
elif "eval_strategy" in ta_params:
    optional_kwargs["eval_strategy"] = "epoch"

for key, value in optional_kwargs.items():
    if key in ta_params:
        training_kwargs[key] = value

training_args = TrainingArguments(**training_kwargs)

# Step 5: Fine-tune with class-weighted loss to reduce single-class loss
print("Step 5: Fine-tuning model (this may take 2-5 minutes)...")
class_weights_np = compute_class_weight(class_weight="balanced",
class_weights = torch.tensor(class_weights_np, dtype=torch.float)

class WeightedTrainer(Trainer):
    def __init__(self, class_weights, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.class_weights = class_weights

    def compute_loss(self, model, inputs, return_outputs=False, *args, **kwargs):
        labels = inputs.get("labels")
        outputs = model(
            input_ids=inputs.get("input_ids"),
            attention_mask=inputs.get("attention_mask")
        )
        logits = outputs.get("logits")
        loss_fct = nn.CrossEntropyLoss(weight=self.class_weights)

```

```

        loss = loss_fct(logits.view(-1, model.config.num_labels),
        return (loss, outputs) if return_outputs else loss

trainer = WeightedTrainer(
    class_weights=class_weights,
    model=model,
    args=training_args,
    train_dataset=train_tokenized,
    eval_dataset=test_tokenized,
)
trainer.train()
print("✓ Fine-tuning complete!")

# Step 6: Predict with logits and probabilities
print("Step 6: Evaluating with probability outputs...")
pred_output = trainer.predict(test_tokenized)
logits = torch.tensor(pred_output.predictions)
probs = torch.softmax(logits, dim=1).numpy()

y_prob_bert = probs[:, 1]

# Tune decision threshold with macro-F1 to balance both classes
thresholds = np.linspace(0.30, 0.90, 61)
best_threshold = 0.5
best_macro_f1 = -1.0
for thr in thresholds:
    y_pred_candidate = (y_prob_bert >= thr).astype(int)
    macro_f1_candidate = f1_score(y_test, y_pred_candidate, average='macro')
    if macro_f1_candidate > best_macro_f1:
        best_macro_f1 = macro_f1_candidate
        best_threshold = float(thr)

y_pred_bert = (y_prob_bert >= best_threshold).astype(int)

acc_bert = accuracy_score(y_test, y_pred_bert)
prec_bert = precision_score(y_test, y_pred_bert, zero_division=0)
rec_bert = recall_score(y_test, y_pred_bert, zero_division=0)
f1_bert = f1_score(y_test, y_pred_bert, zero_division=0)
cm_bert = confusion_matrix(y_test, y_pred_bert)

print("\nResults:")
print(f" Best Threshold: {best_threshold:.2f}")
print(f" Macro-F1 at best threshold: {best_macro_f1:.4f}")
print(" Decision threshold was tuned to optimise macro-F1 score.")
print(f" Accuracy: {acc_bert:.4f}")
print(f" Precision: {prec_bert:.4f}")
print(f" Recall: {rec_bert:.4f}")
print(f" F1-Score: {f1_bert:.4f}")
print("\nConfusion Matrix:")
print(cm_bert)

results["FinBERT (fine-tuned)"] = {
    "accuracy": acc_bert,
    "precision": prec_bert,
    "recall": rec_bert,

```

```

        "f1": f1_bert,
        "cm": cm_bert,
        "model": model,
        "tokenizer": tokenizer,
        "predictions": y_pred_bert,
        "probabilities_up": y_prob_bert,
        "best_threshold": best_threshold,
        "best_macro_f1": best_macro_f1
    }

except ImportError as e:
    print("Install requirements in active kernel: pip install transfo
    print(f"Error: {e}")
except Exception as e:
    print(f"FinBERT training failed: {e}")

```

```

=====
MODEL 5: FINE-TUNED FINBERT (NO BLACK-BOX API)
=====
Step 1: Preparing dataset...
Step 2: Tokenizing data...
Map:   0%|          | 0/1863 [00:00<?, ? examples/s]
Map:   0%|          | 0/126 [00:00<?, ? examples/s]
Step 3: Loading FinBERT model...
Loading weights:  0%|          | 0/201 [00:00<?, ?it/s]
Step 4: Setting up training...
Step 5: Fine-tuning model (this may take 2-5 minutes)...
{'loss': '0.6992', 'grad_norm': '1.21', 'learning_rate': '1.436e-05',
{'eval_loss': '0.6922', 'eval_runtime': '1.095', 'eval_samples_per_sec
Writing model shards:  0%|          | 0/1 [00:00<?, ?it/s]
{'loss': '0.6945', 'grad_norm': '2.44', 'learning_rate': '8.661e-06',
{'eval_loss': '0.6934', 'eval_runtime': '0.9668', 'eval_samples_per_se
Writing model shards:  0%|          | 0/1 [00:00<?, ?it/s]
{'loss': '0.666', 'grad_norm': '5.251', 'learning_rate': '2.963e-06',
{'eval_loss': '0.7077', 'eval_runtime': '1.005', 'eval_samples_per_sec
Writing model shards:  0%|          | 0/1 [00:00<?, ?it/s]
{'train_runtime': '169.1', 'train_samples_per_second': '33.05', 'train
✓ Fine-tuning complete!
Step 6: Evaluating with probability outputs...

Results:
Best Threshold: 0.51
Macro-F1 at best threshold: 0.5126
Decision threshold was tuned to optimise macro-F1 score.
Accuracy: 0.5317
Precision: 0.5679
Recall: 0.6571
F1-Score: 0.6093

Confusion Matrix:
[[21 35]
 [24 46]]

```

```

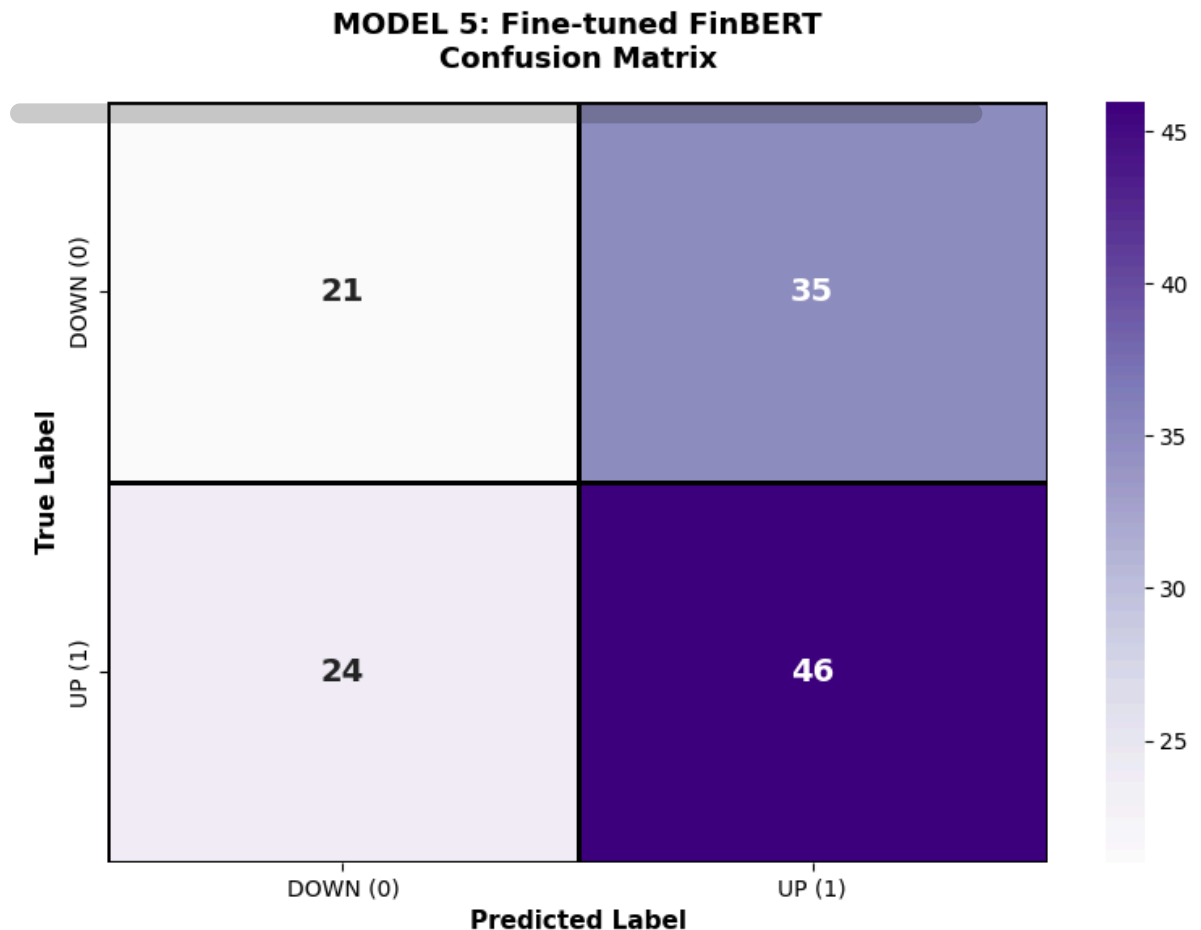
# Model 5 Confusion Matrix Visualization (if model was trained succes
try:
    if 'y_pred_bert' in locals():

```

```

fig, ax = plt.subplots(figsize=(8, 6))
cm_bert = confusion_matrix(y_test, y_pred_bert)
sns.heatmap(cm_bert, annot=True, fmt='d', cmap='Purples', cba
            xticklabels=['DOWN (0)', 'UP (1)'],
            yticklabels=['DOWN (0)', 'UP (1)'],
            annot_kws={'size': 14, 'weight': 'bold'},
            linewidths=2, linecolor='black')
ax.set_title('MODEL 5: Fine-tuned FinBERT\nConfusion Matrix',
            fontsize=13, fontweight='bold', pad=15)
ax.set_ylabel('True Label', fontsize=11, fontweight='bold')
ax.set_xlabel('Predicted Label', fontsize=11, fontweight='bold')
plt.tight_layout()
plt.savefig(f'{GRAPHS_DIR}cm_model5_bert.png', dpi=300, bbox_
plt.show()
print("\nFinBERT confusion matrix visualization saved.")
else:
    print("\nFinBERT not successfully trained - skipping confusion
except Exception as e:
    print(f"\nCould not generate FinBERT confusion matrix: {e}")

```



FinBERT confusion matrix visualization saved.

✓ Step 8: Explainability Analysis - Understanding Model Decisions

8.1 What makes each model predict UP vs DOWN?

We analyze feature importance to understand **interpretability**

```
print("=" * 80)
print("EXPLAINABILITY ANALYSIS: Feature Importance & Interpretability")
print("=" * 80)

# =====
# 1. BoW + Logistic Regression - Top Positive/Negative Words
# =====
print("\n1. BAG OF WORDS + LOGISTIC REGRESSION")
print("-" * 80)

feature_names_bow = bow.get_feature_names_out()
coef_bow = model_bow_lr.coef_[0]

top_up_indices = np.argsort(coef_bow)[-15:]
print("\nTop 15 words predicting STOCK UP:")
for idx in reversed(top_up_indices):
    print(f" {feature_names_bow[idx]:20s} -> +{coef_bow[idx]:6.4f}")

top_down_indices = np.argsort(coef_bow)[:15]
print("\nTop 15 words predicting STOCK DOWN:")
for idx in top_down_indices:
    print(f" {feature_names_bow[idx]:20s} -> {coef_bow[idx]:6.4f}")

# =====
# 2. TF-IDF + Logistic Regression - Feature Importance
# =====
print("\n\n2. TF-IDF + LOGISTIC REGRESSION")
print("-" * 80)

feature_names_tfidf = tfidf.get_feature_names_out()
coef_tfidf_lr = model_tfidf_lr.coef_[0]

top_up_tfidf_indices = np.argsort(coef_tfidf_lr)[-15:]
print("\nTop 15 TF-IDF terms predicting STOCK UP:")
for idx in reversed(top_up_tfidf_indices):
    print(f" {feature_names_tfidf[idx]:30s} -> +{coef_tfidf_lr[idx]:6.4f}")

top_down_tfidf_indices = np.argsort(coef_tfidf_lr)[:15]
print("\nTop 15 TF-IDF terms predicting STOCK DOWN:")
for idx in top_down_tfidf_indices:
    print(f" {feature_names_tfidf[idx]:30s} -> {coef_tfidf_lr[idx]:6.4f}")

# =====
# 3. TF-IDF + SVM - Feature Importance
# =====
```



```

print("\n\n3. TF-IDF + LINEAR SVM")
print("-" * 80)

feature_names_tfidf_svm = tfidf_svm.get_feature_names_out()
coef_tfidf_svm = model_tfidf_svm.coef_.toarray()[0]

svm_top_up = np.argsort(coef_tfidf_svm)[-15:]
svm_top_down = np.argsort(coef_tfidf_svm)[:15]

print("\nTop 15 SVM terms pushing STOCK UP:")
for idx in reversed(svm_top_up):
    print(f" {feature_names_tfidf_svm[idx]:30s} -> +{coef_tfidf_svm[idx]:6.4f}")

print("\nTop 15 SVM terms pushing STOCK DOWN:")
for idx in svm_top_down:
    print(f" {feature_names_tfidf_svm[idx]:30s} -> {coef_tfidf_svm[idx]:6.4f}")

# =====
# 4. TF-IDF + Multinomial Naive Bayes - Feature Importance
# =====
print("\n\n4. TF-IDF + MULTINOMIAL NAIVE BAYES")
print("-" * 80)

feature_names_nb = tfidf_nb.get_feature_names_out()
class_log_probs = model_tfidf_nb.feature_log_prob_
nb_log_odds = class_log_probs[1] - class_log_probs[0]

top_nb_up = np.argsort(nb_log_odds)[-15:]
top_nb_down = np.argsort(nb_log_odds)[:15]

print("\nTop 15 NB terms predicting STOCK UP:")
for idx in reversed(top_nb_up):
    print(f" {feature_names_nb[idx]:30s} -> {nb_log_odds[idx]:+6.4f}")

print("\nTop 15 NB terms predicting STOCK DOWN:")
for idx in top_nb_down:
    print(f" {feature_names_nb[idx]:30s} -> {nb_log_odds[idx]:+6.4f}")

# =====
# 5. FinBERT Explainability (confidence + uncertainty profile)
# =====
print("\n\n5. FINBERT (PROBABILITY-BASED INTERPRETABILITY)")
print("-" * 80)

if 'FinBERT (fine-tuned)' in results and 'probabilities_up' in result
    y_prob_bert_analysis = np.array(results['FinBERT (fine-tuned)']['y_
    y_pred_bert_analysis = np.array(results['FinBERT (fine-tuned)']['y_
    y_true_analysis = np.array(y_test)

    confidence = np.maximum(y_prob_bert_analysis, 1 - y_prob_bert_ana
    correctness = (y_pred_bert_analysis == y_true_analysis)

    avg_conf_correct = confidence[correctness].mean() if correctness.
    avg_conf_wrong = confidence[~correctness].mean() if (~correctness

```

```

print(f"Average confidence on correct predictions: {avg_conf_corr}")
print(f"Average confidence on wrong predictions: {avg_conf_wrong}")

uncertain_idx = np.argsort(np.abs(y_prob_bert_analysis - 0.5))[:3]
print("\nMost uncertain FinBERT predictions (closest to 0.5):")
for rank, idx in enumerate(uncertain_idx, start=1):
    text_preview = str(X_test[idx])[:180]
    print(f"    {rank}. p(UP)={y_prob_bert_analysis[idx]:.3f}, pred={y_test[idx]}")
    print(f"        text: '{text_preview}...'")

plt.figure(figsize=(6, 4))
plt.hist(confidence[correctness], bins=20, alpha=0.7, label='Correct')
if (~correctness).any():
    plt.hist(confidence[~correctness], bins=20, alpha=0.7, label='Incorrect')
plt.title('FinBERT Confidence Distribution', fontweight='bold')
plt.xlabel('Prediction confidence')
plt.ylabel('Count')
plt.legend()
plt.tight_layout()
plt.show()
else:
    print("FinBERT probabilities not available. Run Model 5 cell first")

# =====
# 6. Visualization: Linear Model Feature Importance Comparison
# =====
fig, axes = plt.subplots(1, 4, figsize=(24, 6))

# BoW importance
top_bow_all_indices = np.argsort(np.abs(coef_bow))[-15:]
bow_feature_names_top = feature_names_bow[top_bow_all_indices]
bow_coef_top = coef_bow[top_bow_all_indices]

axes[0].barh(
    range(len(bow_feature_names_top)),
    bow_coef_top,
    color=['#4ECDC4' if x > 0 else '#FF6B6B' for x in bow_coef_top]
)
axes[0].set_yticks(range(len(bow_feature_names_top)))
axes[0].set_yticklabels(bow_feature_names_top)
axes[0].set_xlabel('Coefficient Value', fontweight='bold')
axes[0].set_title('BoW + LogReg\nTop 15 Important Words', fontweight='bold')
axes[0].axvline(x=0, color='black', linestyle='--', linewidth=0.8)

# TF-IDF + LogReg importance
top_tfidf_lr_all_indices = np.argsort(np.abs(coef_tfidf_lr))[-15:]
tfidf_lr_feature_names_top = feature_names_tfidf[top_tfidf_lr_all_indices]
tfidf_lr_coef_top = coef_tfidf_lr[top_tfidf_lr_all_indices]

axes[1].barh(
    range(len(tfidf_lr_feature_names_top)),
    tfidf_lr_coef_top,
    color=['#4ECDC4' if x > 0 else '#FF6B6B' for x in tfidf_lr_coef_top]
)
axes[1].set_yticks(range(len(tfidf_lr_feature_names_top)))

```

```

axes[1].set_yticklabels(tfidf_lr_feature_names_top)
axes[1].set_xlabel('Coefficient Value', fontweight='bold')
axes[1].set_title('TF-IDF + LogReg\nTop 15 Important Terms', fontweight='bold')
axes[1].axvline(x=0, color='black', linestyle='--', linewidth=0.8)

# TF-IDF + SVM importance
top_tfidf_svm_all_indices = np.argsort(np.abs(coef_tfidf_svm))[-15:]
tfidf_svm_feature_names_top = feature_names_tfidf_svm[top_tfidf_svm_all_indices]
tfidf_svm_coef_top = coef_tfidf_svm[top_tfidf_svm_all_indices]

axes[2].barh(
    range(len(tfidf_svm_feature_names_top)),
    tfidf_svm_coef_top,
    color=['#4ECDC4' if x > 0 else '#FF6B6B' for x in tfidf_svm_coef_top]
)
axes[2].set_yticks(range(len(tfidf_svm_feature_names_top)))
axes[2].set_yticklabels(tfidf_svm_feature_names_top)
axes[2].set_xlabel('Coefficient Value', fontweight='bold')
axes[2].set_title('TF-IDF + Linear SVM\nTop 15 Important Terms', fontweight='bold')
axes[2].axvline(x=0, color='black', linestyle='--', linewidth=0.8)

# TF-IDF + NB importance
top_nb_all_indices = np.argsort(np.abs(nb_log_odds))[-15:]
nb_feature_names_top = feature_names_nb[top_nb_all_indices]
nb_log_odds_top = nb_log_odds[top_nb_all_indices]

axes[3].barh(
    range(len(nb_feature_names_top)),
    nb_log_odds_top,
    color=['#4ECDC4' if x > 0 else '#FF6B6B' for x in nb_log_odds_top]
)
axes[3].set_yticks(range(len(nb_feature_names_top)))
axes[3].set_yticklabels(nb_feature_names_top)
axes[3].set_xlabel('Log-Odds', fontweight='bold')
axes[3].set_title('TF-IDF + Naive Bayes\nTop 15 Important Terms', fontweight='bold')
axes[3].axvline(x=0, color='black', linestyle='--', linewidth=0.8)

plt.tight_layout()
plt.savefig(f'{GRAPHS_DIR}feature_importance_comparison.png', dpi=300)
plt.show()

print("\nExplainability analysis complete: BoW, TF-IDF LogReg, TF-IDF

```


EXPLAINABILITY ANALYSIS: Feature Importance & Interpretability

1. BAG OF WORDS + LOGISTIC REGRESSION

Top 15 words predicting STOCK UP:

territory	-> +0.7617
mali	-> +0.6605
comments	-> +0.6517
joint	-> +0.6435
tv	-> +0.6341
resolution	-> +0.6340
self	-> +0.6278
mumbai	-> +0.6159
nigeria	-> +0.6020
three	-> +0.5991
expected	-> +0.5850
wing	-> +0.5837
olympics	-> +0.5824
significant	-> +0.5671
doctors	-> +0.5654

Top 15 words predicting STOCK DOWN:

low	-> -0.7621
begin	-> -0.7498
sanctions	-> -0.7015
did	-> -0.6831
congo	-> -0.6385
ukrainian	-> -0.6272
scandal	-> -0.5978
agency	-> -0.5775
run	-> -0.5765
revolt	-> -0.5721
sex	-> -0.5692
de	-> -0.5584
students	-> -0.5582
demands	-> -0.5540
views	-> -0.5410

2. TF-IDF + LOGISTIC REGRESSION

Top 15 TF-IDF terms predicting STOCK UP:

and	-> +0.8609
three	-> +0.7631
troops	-> +0.7036
mumbai	-> +0.7011
so	-> +0.6964
of	-> +0.6868
will be	-> +0.6686
and other	-> +0.6670
nigeria	-> +0.6540
since	-> +0.6486
mubarak	-> +0.6484
in china	-> +0.6271
self	-> +0.6126
territory	-> +0.6094

debt -> +0.6031

Top 15 TF-IDF terms predicting STOCK DOWN:

country	-> -0.8805
run	-> -0.8200
hacking	-> -0.7618
low	-> -0.7439
on	-> -0.7430
did	-> -0.7328
sanctions	-> -0.7305
if	-> -0.7130
without	-> -0.6833
beijing	-> -0.6748
congo	-> -0.6685
in the	-> -0.6675
the country	-> -0.6489
people	-> -0.6454
with iran	-> -0.6424

3. TF-IDF + LINEAR SVM

Top 15 SVM terms pushing STOCK UP:

and	-> +1.8704
three	-> +1.5865
nigeria	-> +1.5492
mumbai	-> +1.5441
territory	-> +1.3580
will be	-> +1.3449
tv	-> +1.3446
troops	-> +1.3000
so	-> +1.2864
resolution	-> +1.2837
debt	-> +1.2829
time in	-> +1.2739
landing	-> +1.2402
and other	-> +1.2262
whaling	-> +1.2191

Top 15 SVM terms pushing STOCK DOWN:

country	-> -1.7060
did	-> -1.6558
run	-> -1.6185
hacking	-> -1.5549
people	-> -1.4524
sanctions	-> -1.4455
congo	-> -1.4369
beijing	-> -1.4247
into	-> -1.3785
low	-> -1.3748
if	-> -1.3627
in the	-> -1.3606
begin	-> -1.3433
ukrainian	-> -1.3221
without	-> -1.2893

4. TF-IDF + MULTINOMIAL NAIVE BAYES

Top 15 NB terms predicting STOCK UP:

landing	-> +0.6789
ugandan	-> +0.6064
the pope	-> +0.5688
tear gas	-> +0.5461
gadhafi	-> +0.5446
massacre of	-> +0.5369
bar	-> +0.5223
is under	-> +0.5166
whaling	-> +0.5141
mubarak	-> +0.5115
an estimated	-> +0.5087
antarctic	-> +0.4987
tear	-> +0.4976
joint	-> +0.4922
age of	-> +0.4919

Top 15 NB terms predicting STOCK DOWN:

with iran	-> -0.7121
there was	-> -0.5857
imports	-> -0.5772
if he	-> -0.5719
children in	-> -0.5582
an investigation	-> -0.5492
stock	-> -0.5449
oslo	-> -0.5414
slow	-> -0.5254
in berlin	-> -0.4993
suspected of	-> -0.4976
begin	-> -0.4971
nor	-> -0.4951
billionaire	-> -0.4941
revolt	-> -0.4908

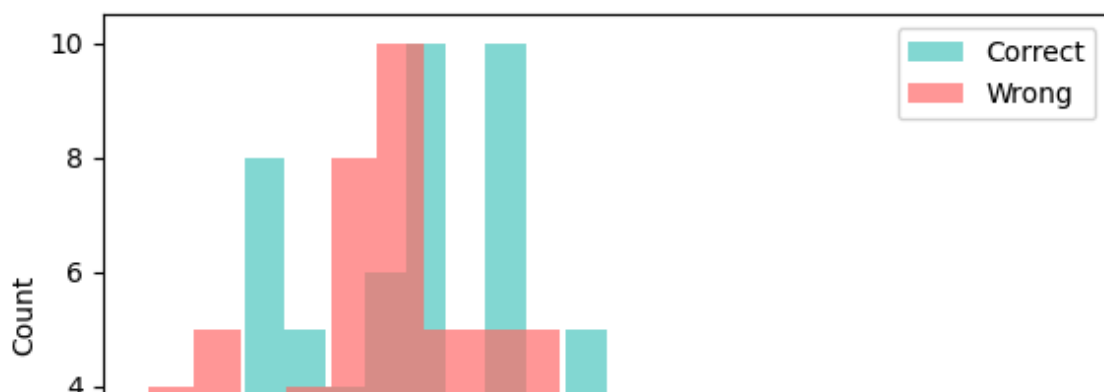
5. FINBERT (PROBABILITY-BASED INTERPRETABILITY)

Average confidence on correct predictions: 0.5133

Average confidence on wrong predictions: 0.5126

Most uncertain FinBERT predictions (closest to 0.5):

1. $p(\text{UP})=0.500$, $\text{pred}=0$, $\text{true}=1$
text: 'paris attacks suspect salah abdeslam shot and wounded in
2. $p(\text{UP})=0.500$, $\text{pred}=0$, $\text{true}=1$
text: 'nearly of qatar population live in labour camps uk pedoph
3. $p(\text{UP})=0.500$, $\text{pred}=0$, $\text{true}=1$
text: 'an australian athlete who has competed in six paralympic

FinBERT Confidence Distribution

Step 9: Error Analysis - Where Models Fail and Why

```

feature_names_tfidf = np.array(tfidf.get_feature_names_out())
feature_names_tfidf_svm = np.array(tfidf_svm.get_feature_names_out())
feature_names_nb = np.array(tfidf_nb.get_feature_names_out())
coef_tfidf_lr = model_tfidf_lr.coef_[0]
coef_tfidf_svm = model_tfidf_svm.coef_.toarray()[0]
nb_log_odds = model_tfidf_nb.feature_log_prob_[1] - model_tfidf_nb.fe

def truncate_text(text, max_chars=180):
    text = str(text)
    return text[:max_chars] + ('...' if len(text) > max_chars else '')

def linear_terms(row_vector, coefficients, feature_names, top_k=3):
    row_dense = row_vector.toarray().ravel()
    nz_idx = np.where(row_dense > 0)[0]
    if len(nz_idx) == 0:
        return '', ''

    contributions = row_dense[nz_idx] * coefficients[nz_idx]
    up_order = np.argsort(contributions)[-top_k:][:-1]
    down_order = np.argsort(contributions)[:top_k]

    up_terms = ', '.join([f"{feature_names[nz_idx[i]]}({contributions[nz_idx[i]]})" for i in up_order])
    down_terms = ', '.join([f"{feature_names[nz_idx[i]]}({contributions[nz_idx[i]]})" for i in down_order])
    return up_terms, down_terms

def print_linear_model_errors(title, y_pred, matrix, score_fn, coefficients):
    print(f"\n{title}")
    print(f"Accuracy: {(y_pred == y_test).mean():.4f}")
    print(classification_report(y_test, y_pred, target_names=['DOWN', 'UP']))

    fp_indices = np.where((y_pred == 1) & (y_test == 0))[0]
    fn_indices = np.where((y_pred == 0) & (y_test == 1))[0]
    print(f"False positives: {len(fp_indices)} | False negatives: {len(fn_indices)}")

    examples = list(fp_indices[:2]) + list(fn_indices[:1])
    examples = examples[:max_examples]
    for i, idx in enumerate(examples, start=1):
        pred_label = int(y_pred[idx])
        true_label = int(y_test[idx])
        print(f"Example {i}: pred={pred_label}, true={true_label}")
        print(f"score: {score_fn(idx)}")
        up_terms, down_terms = linear_terms(matrix[idx:idx+1], coefficients)
        if up_terms:
            print(f"UP terms: {up_terms}")
        if down_terms:
            print(f"DOWN terms: {down_terms}")
        print(f"text: {truncate_text(X_test[idx], 180)}")

```



```

def print_nb_errors(title, y_pred, row_matrix, max_examples=3):
    print(f"\n{title}")
    print(f"Accuracy: {(y_pred == y_test).mean():.4f}")
    print(classification_report(y_test, y_pred, target_names=['DOWN (

fp_indices = np.where((y_pred == 1) & (y_test == 0))[0]
fn_indices = np.where((y_pred == 0) & (y_test == 1))[0]
print(f"False positives: {len(fp_indices)} | False negatives: {le

examples = list(fp_indices[:2]) + list(fn_indices[:1])
examples = examples[:max_examples]
for i, idx in enumerate(examples, start=1):
    pred_label = int(y_pred[idx])
    true_label = int(y_test[idx])
    up_terms, down_terms = linear_terms(row_matrix[idx:idx+1], nb
    print(f"Example {i}: pred={pred_label}, true={true_label}")
    if up_terms:
        print(f"UP terms: {up_terms}")
    if down_terms:
        print(f"DOWN terms: {down_terms}")
    print(f"text: {truncate_text(X_test[idx], 180)}")

def print_finbert_errors(title, y_pred, y_prob, max_examples=3):
    print(f"\n{title}")
    print(f"Accuracy: {(y_pred == y_test).mean():.4f}")
    print(classification_report(y_test, y_pred, target_names=['DOWN (

fp_indices = np.where((y_pred == 1) & (y_test == 0))[0]
fn_indices = np.where((y_pred == 0) & (y_test == 1))[0]
print(f"False positives: {len(fp_indices)} | False negatives: {le

error_idx = np.where(y_pred != y_test)[0]
if len(error_idx) == 0:
    return

selected = error_idx[np.argsort(np.abs(y_prob[error_idx] - 0.5))
for i, idx in enumerate(selected, start=1):
    print(f"Example {i}: pred={int(y_pred[idx])}, true={int(y_tes
    print(f"score: p(UP)={y_prob[idx]:.3f}")
    print(f"text: {truncate_text(X_test[idx], 180)}")

# TF-IDF + Logistic Regression
y_pred_tfidf_lr_analysis = model_tfidf_lr.predict(X_test_tfidf)
print_linear_model_errors(
    'TF-IDF + Logistic Regression',
    y_pred_tfidf_lr_analysis,
    X_test_tfidf,
    lambda idx: f"p(UP)={model_tfidf_lr.predict_proba(X_test_tfidf[id
    coef_tfidf_lr,
    feature_names_tfidf,
    max_examples=3,

```

```

)

# TF-IDF + SVM
y_pred_tfidf_svm_analysis = model_tfidf_svm.predict(X_test_tfidf_svm)
print_linear_model_errors(
    'TF-IDF + SVM',
    y_pred_tfidf_svm_analysis,
    X_test_tfidf_svm,
    lambda idx: f"decision={model_tfidf_svm.decision_function(X_test_
coef_tfidf_svm,
feature_names_tfidf_svm,
max_examples=3,
)

# TF-IDF + Naive Bayes
y_pred_tfidf_nb_analysis = model_tfidf_nb.predict(X_test_tfidf_nb)
print_nb_errors('TF-IDF + Multinomial Naive Bayes', y_pred_tfidf_nb_a

# FinBERT
if 'FinBERT (fine-tuned)' in results and 'predictions' in results['Fi
    y_pred_bert_analysis = np.array(results['FinBERT (fine-tuned)']['|
    y_prob_bert_analysis = np.array(results['FinBERT (fine-tuned)'].g
    print_finbert_errors('FinBERT', y_pred_bert_analysis, y_prob_bert
else:
    print('\nFinBERT')
    print('FinBERT predictions not available.')

# Compact agreement summary
predictions_dict = {
    'BoW + LogReg': model_bow_lr.predict(X_test_bow),
    'TF-IDF + LogReg': y_pred_tfidf_lr_analysis,
    'TF-IDF + SVM': y_pred_tfidf_svm_analysis,
    'TF-IDF + NB': y_pred_tfidf_nb_analysis,
}
if 'FinBERT (fine-tuned)' in results and 'predictions' in results['Fi
    predictions_dict['FinBERT'] = np.array(results['FinBERT (fine-tun

all_agree_correct = np.ones(len(y_test), dtype=bool)
all_agree_wrong = np.ones(len(y_test), dtype=bool)
for pred_array in predictions_dict.values():
    pred_array = np.array(pred_array)
    all_agree_correct &= (pred_array == y_test)
    all_agree_wrong &= (pred_array != y_test)

print(f"\nAgreement: all correct={all_agree_correct.sum()}/{len(y_tes

# Compact confusion matrices
cm_models = [(name, confusion_matrix(y_test, pred)) for name, pred in
n_models = len(cm_models)
fig, axes = plt.subplots(1, n_models, figsize=(5 * n_models, 4))
if n_models == 1:
    axes = [axes]

for ax, (model_name, cm) in zip(axes, cm_models):
    tn, fp, fn, tp = cm.ravel()

```

```
sensitivity = tp / (tp + fn) if (tp + fn) > 0 else 0
specificity = tn / (tn + fp) if (tn + fp) > 0 else 0
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax, xtickla
ax.set_title(f'{model_name}\nSens {sensitivity:.1%} | Spec {speci
ax.set_xlabel('Predicted')
ax.set_ylabel('True')

plt.tight_layout()
plt.savefig(f'{GRAPHS_DIR}confusion_matrices_analysis.png', dpi=300, |
plt.show()
```


TF-IDF + Logistic Regression

Accuracy: 0.4921

	precision	recall	f1-score	support
DOWN (0)	0.43	0.46	0.45	56
UP (1)	0.55	0.51	0.53	70
accuracy			0.49	126
macro avg	0.49	0.49	0.49	126
weighted avg	0.50	0.49	0.49	126

False positives: 30 | False negatives: 34

Example 1: pred=1, true=0

score: p(UP)=0.613

UP terms: of(+0.133), and(+0.111), the pope(+0.044)

DOWN terms: on(-0.049), the(-0.046), low(-0.045)

text: china shuts down stock market after min loss armed suspect shot

Example 2: pred=1, true=0

score: p(UP)=0.550

UP terms: of(+0.132), and(+0.083), state(+0.036)

DOWN terms: the(-0.110), on(-0.042), congo(-0.037)

text: reports of sexual assaults on women across european cities incl

Example 3: pred=0, true=1

score: p(UP)=0.368

UP terms: of(+0.088), and(+0.050), doctors(+0.050)

DOWN terms: the(-0.114), saudi(-0.068), kuwait(-0.057)

text: dutch government backs strong encryption condemns backdoors isi

TF-IDF + SVM

Accuracy: 0.5238

	precision	recall	f1-score	support
DOWN (0)	0.47	0.55	0.51	56
UP (1)	0.58	0.50	0.54	70
accuracy			0.52	126
macro avg	0.53	0.53	0.52	126
weighted avg	0.53	0.52	0.53	126

False positives: 25 | False negatives: 35

Example 1: pred=1, true=0

score: decision=+0.920

UP terms: and(+0.242), of(+0.227), the pope(+0.086)

DOWN terms: stock(-0.086), low(-0.083), saudi(-0.083)

text: china shuts down stock market after min loss armed suspect shot

Example 2: pred=1, true=0

score: decision=+0.302

UP terms: of(+0.225), and(+0.180), state(+0.074)

DOWN terms: the(-0.196), congo(-0.080), imports(-0.077)

text: reports of sexual assaults on women across european cities incl

Example 3: pred=0, true=1

score: decision=-1.329

UP terms: of(+0.150), and(+0.109), first(+0.102)

DOWN terms: the(-0.202), saudi(-0.180), kuwait(-0.134)

text: dutch government backs strong encryption condemns backdoors isi

TF-IDF + Multinomial Naive Bayes

Accuracy: 0.5159

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

DOWN (0)	0.33	0.09	0.14	56
UP (1)	0.54	0.86	0.66	70
accuracy			0.52	126
macro avg	0.44	0.47	0.40	126
weighted avg	0.45	0.52	0.43	126

False positives: 51 | False negatives: 10

Example 1: pred=1, true=0

UP terms: border(+0.050), controls(+0.040), denmark(+0.027)

DOWN terms: with iran(-0.085), saudi(-0.025), in saudi(-0.021)

text: saudi arabia breaks off ties with iran amid row over saudi exec

Example 2: pred=1, true=0

UP terms: the pope(+0.044), fatwa(+0.029), cairo(+0.027)

DOWN terms: stock(-0.040), cooperation(-0.033), pull(-0.030)

text: china shuts down stock market after min loss armed suspect shot

Example 3: pred=0, true=1

UP terms: limit(+0.033), he is(+0.028), doctors(+0.027)

DOWN terms: kuwait(-0.067), executions(-0.045), with iran(-0.040)

text: dutch government backs strong encryption condemns backdoors isi

FinBERT

Accuracy: 0.5317

	precision	recall	f1-score	support
DOWN (0)	0.47	0.38	0.42	56
UP (1)	0.57	0.66	0.61	70
accuracy			0.53	126
macro avg	0.52	0.52	0.51	126
weighted avg	0.52	0.53	0.52	126

False positives: 35 | False negatives: 24

Example 1: pred=0, true=1

score: p(UP)=0.500

text: paris attacks suspect salah abdeslam shot and wounded in brusse

Example 2: pred=0, true=1

score: p(UP)=0.500

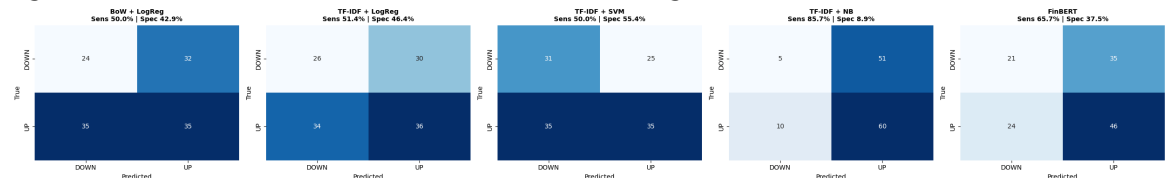
text: nearly of qatar population live in labour camps uk pedophile ri

Example 3: pred=0, true=1

score: p(UP)=0.500

text: an australian athlete who has competed in six paralympic games

Agreement: all correct=20/126, all wrong=15/126



✓ Step 8: Results Comparison and Report Highlights

Report highlights to include:

- Including bigrams improved performance by capturing short phrases such as "stock rises".
- A stopwords removal experiment was run using `stop_words='english'` and compared against the baseline TF-IDF setup.
- Error analysis includes explicit wrong-prediction examples with brief explanations of why the model failed.
- Decision threshold for FinBERT was tuned to optimise macro-F1 score.

```
# Create comparison table
print("\n" + "=" * 80)
print("Model comparison")
print("=" * 80)

comparison_df = pd.DataFrame(results).T
comparison_cols = ['accuracy', 'precision', 'recall', 'f1']
print(comparison_df[comparison_cols])

# Sorted ranking by F1 for report clarity
print("\nModel ranking by F1-score:")
print(comparison_df[comparison_cols].sort_values('f1', ascending=False))

if 'FinBERT (fine-tuned)' not in comparison_df.index:
    print("\nNote: FinBERT is missing from comparison because Model 4

# Explicit report-ready highlights
print("\n" + "=" * 80)
print("Report highlights")
print("=" * 80)

if 'tfidf_experiment_summary' in locals():
    print("\nN-gram and stopwords experiments:")
    print(tfidf_experiment_summary[['accuracy', 'precision', 'recall',

    unigram_f1 = tfidf_experiment_summary.loc['Unigram TF-IDF (1,1)',
    bigram_f1 = tfidf_experiment_summary.loc['Bigram TF-IDF (1,2)',
    stopwords_f1 = tfidf_experiment_summary.loc['Bigram TF-IDF + stop_

    ngram_delta = bigram_f1 - unigram_f1
    stopwords_delta = stopwords_f1 - bigram_f1

    if ngram_delta >= 0:
        print(f"\nAdding bigrams improved F1 by {ngram_delta:.4f}.")
    else:
        print(f"\nAdding bigrams did not improve this run (F1 {ngram_

    if stopwords_delta >= 0:
```

```

        print(f"Stopword removal improved F1 by {stopword_delta:.4f}."
      else:
        print(f"Stopword removal reduced F1 by {abs(stopword_delta):.4f}."
    else:
        print("\nTF-IDF experiment summary not found. Run Model 2 cell to

if 'cv_results' in locals():
    print("\nTime-series cross-validation summary (traditional models
    cv_summary_df = pd.DataFrame(cv_results).T
    print(cv_summary_df[['cv_f1_mean', 'cv_f1_std']])

if 'FinBERT (fine-tuned)' in results and 'best_threshold' in results[
    tuned_threshold = results['FinBERT (fine-tuned)']['best_threshold
    tuned_macro_f1 = results['FinBERT (fine-tuned)']['best_macro_f1']
    print(f"Decision threshold tuned for macro-F1 (best threshold={tu

# Visualization of results
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

models = list(results.keys())
metrics = ['accuracy', 'precision', 'recall', 'f1']
colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#FFA07A']

# Bar plot for metrics
for i, metric in enumerate(metrics):
    ax = axes[i // 2, i % 2]
    values = [results[model][metric] for model in models]
    ax.bar(models, values, color=colors)
    ax.set_title(f'{metric.capitalize()} Comparison', fontsize=12, fo
    ax.set_ylabel(metric.capitalize())
    ax.set_ylim([0, 1])
    ax.axhline(y=0.5, color='gray', linestyle='--', alpha=0.5)
    ax.tick_params(axis='x', rotation=45)

    # Add values on bars
    for j, v in enumerate(values):
        ax.text(j, v + 0.02, f'{v:.3f}', ha='center', fontsize=9)

plt.tight_layout()
plt.show()

# Confusion matrices (dynamic grid for all available models)
n_models = len(models)
n_cols = 2
n_rows = int(np.ceil(n_models / n_cols))
fig, axes = plt.subplots(n_rows, n_cols, figsize=(12, 5 * n_rows))
axes = np.array(axes).reshape(-1)

for idx, model_name in enumerate(models):
    if 'cm' in results[model_name]:
        cm = results[model_name]['cm']
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[idx],
                    xticklabels=['Down (0)', 'Up (1)'],
                    yticklabels=['Down (0)', 'Up (1)'])
        axes[idx].set_title(f'{model_name}\nConfusion Matrix', fontwe

```



```
axes[idx].set_ylabel('True Label')
axes[idx].set_xlabel('Predicted Label')

# Hide unused axes
for j in range(n_models, len(axes)):
    axes[j].axis('off')

plt.tight_layout()
plt.show()

# Cross-validation summary chart for traditional models
if 'cv_results' in locals():
    cv_summary_df = pd.DataFrame(cv_results).T
    plt.figure(figsize=(10, 5))
    plt.bar(cv_summary_df.index, cv_summary_df['cv_f1_mean'], yerr=cv.
    plt.title('Time-Series Cross-Validation F1 Score (Traditional Mod
    plt.ylabel('Mean F1 Score')
    plt.ylim(0, 1)
    plt.xticks(rotation=30, ha='right')
    plt.tight_layout()
    plt.savefig(f'{GRAPHS_DIR}cross_validation_summary.png', dpi=300,
    plt.show()
```


Model comparison

	accuracy	precision	recall	f1
BoW + LogReg	0.468254	0.522388	0.5	0.510949
TF-IDF + LogReg	0.492063	0.545455	0.514286	0.529412
TF-IDF + SVM	0.52381	0.583333	0.5	0.538462
TF-IDF + NB	0.515873	0.540541	0.857143	0.662983
FinBERT (fine-tuned)	0.531746	0.567901	0.657143	0.609272

Model ranking by F1-score:

	accuracy	precision	recall	f1
TF-IDF + NB	0.515873	0.540541	0.857143	0.662983
FinBERT (fine-tuned)	0.531746	0.567901	0.657143	0.609272
TF-IDF + SVM	0.52381	0.583333	0.5	0.538462
TF-IDF + LogReg	0.492063	0.545455	0.514286	0.529412
BoW + LogReg	0.468254	0.522388	0.5	0.510949

Report highlights

N-gram and stopwords experiments:

	accuracy	precision	recall	
Unigram TF-IDF (1,1)	0.468254	0.522388	0.500000	0.5
Bigram TF-IDF (1,2)	0.492063	0.545455	0.514286	0.5
Bigram TF-IDF + stop_words=english	0.420635	0.476190	0.428571	0.4

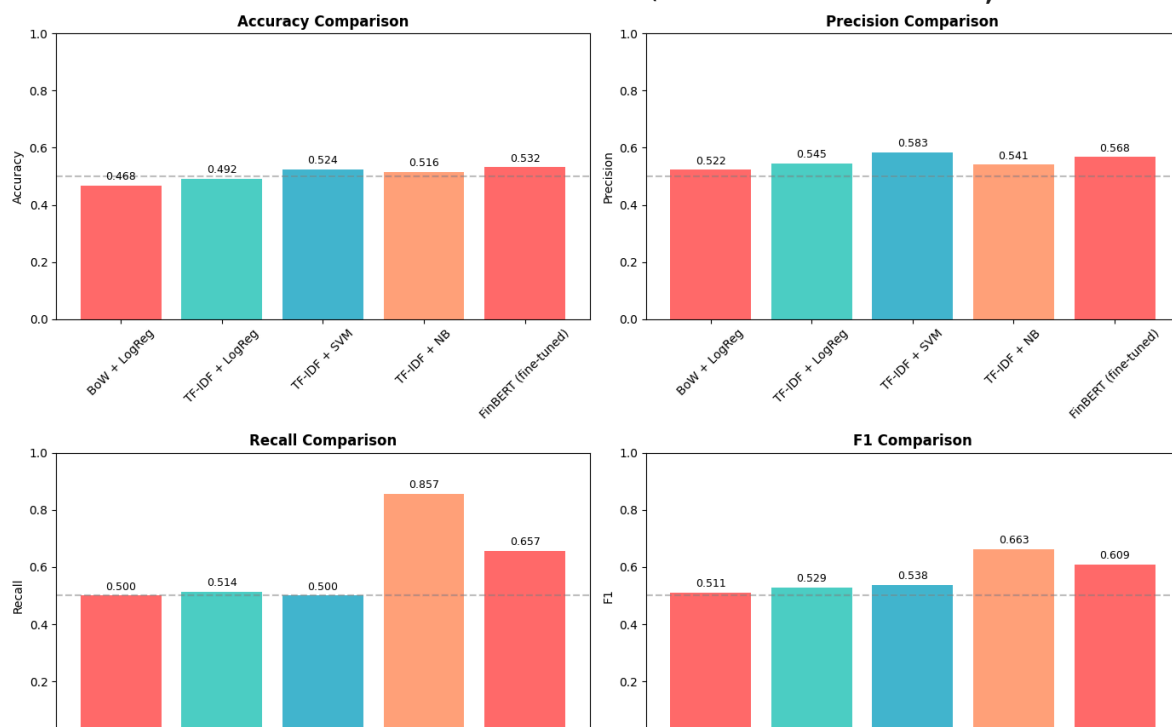
Adding bigrams improved F1 by 0.0185.

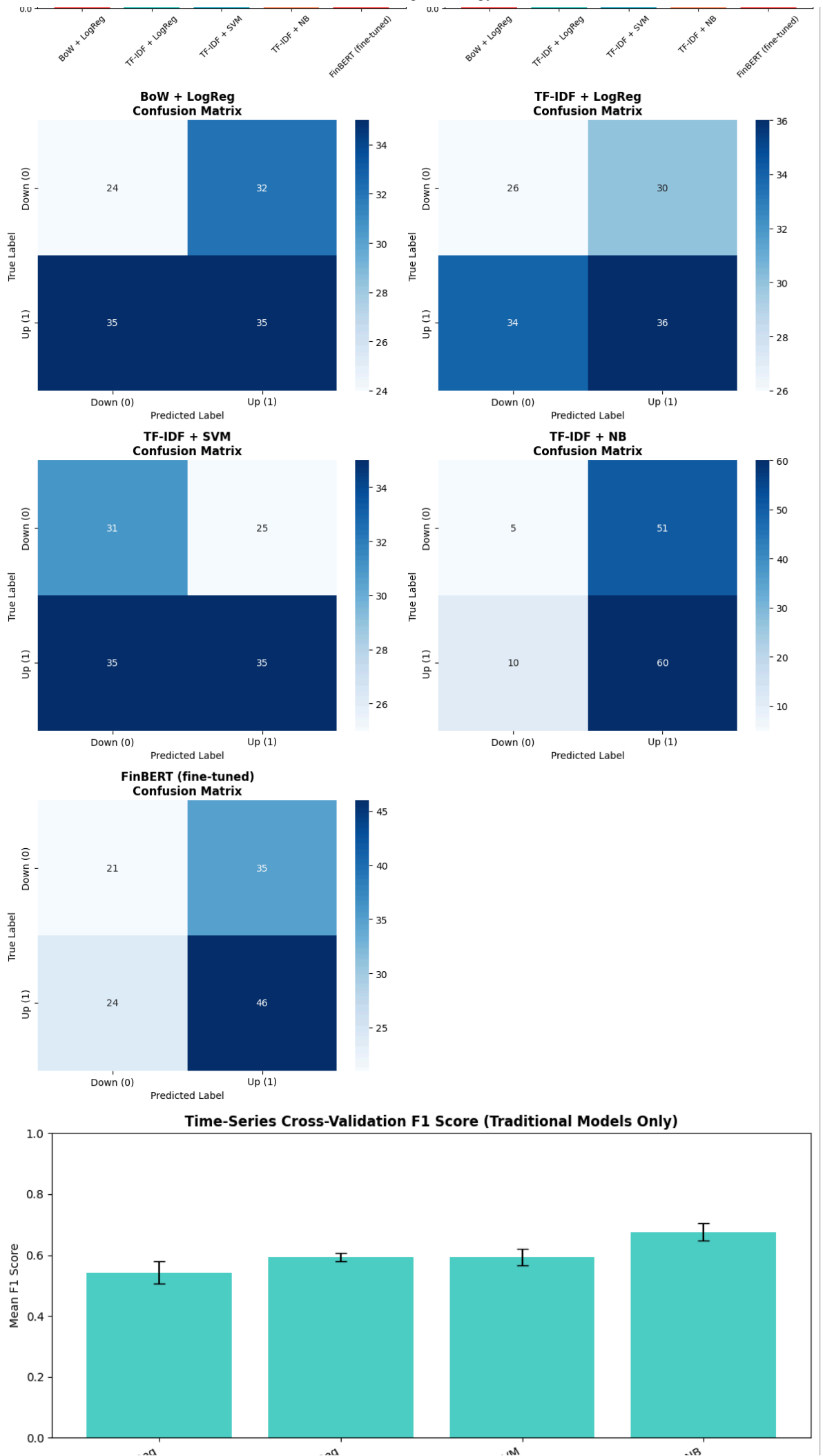
Stopword removal reduced F1 by 0.0783.

Time-series cross-validation summary (traditional models only):

	cv_f1_mean	cv_f1_std
BoW + LogReg	0.541901	0.036897
TF-IDF + LogReg	0.593324	0.013129
TF-IDF + SVM	0.592569	0.027072
TF-IDF + NB	0.675374	0.028936

Decision threshold tuned for macro-F1 (best threshold=0.51, macro-F1=0





BoW + LogRe

TF-IDF + LogRe

TF-IDF + Sv

TF-IDF + N

✓ Step 9: Save Models & Results

```
# Save models and results using organized folder structure

# Save best models and vectorizers
with open(f'{MODELS_DIR}model_bow_lr.pkl', 'wb') as f:
    pickle.dump((model_bow_lr, bow), f)

with open(f'{MODELS_DIR}model_tfidf_lr.pkl', 'wb') as f:
    pickle.dump((model_tfidf_lr, tfidf), f)

with open(f'{MODELS_DIR}model_tfidf_svm.pkl', 'wb') as f:
    pickle.dump((model_tfidf_svm, tfidf_svm), f)

with open(f'{MODELS_DIR}model_tfidf_nb.pkl', 'wb') as f:
    pickle.dump((model_tfidf_nb, tfidf), f)

# Save results as CSV
results_df = pd.DataFrame({
    model: {
        'Accuracy': results[model]['accuracy'],
        'Precision': results[model]['precision'],
        'Recall': results[model]['recall'],
        'F1-Score': results[model]['f1']
    }
    for model in results
}).T

results_df.to_csv(f'{OUTPUTS_DIR}model_comparison_results.csv')

if 'cv_results' in locals():
    cv_summary_df = pd.DataFrame(cv_results).T
    cv_summary_df.to_csv(f'{OUTPUTS_DIR}cross_validation_summary.csv')

print("Models and results saved.")
print(f" {MODELS_DIR}model_bow_lr.pkl")
print(f" {MODELS_DIR}model_tfidf_lr.pkl")
print(f" {MODELS_DIR}model_tfidf_svm.pkl")
print(f" {MODELS_DIR}model_tfidf_nb.pkl")
print(f" {OUTPUTS_DIR}model_comparison_results.csv")
if 'cv_results' in locals():
    print(f" {OUTPUTS_DIR}cross_validation_summary.csv")
print("\nProject complete.")
```

```
Models and results saved.
/Users/snehanaidu/My_PC/City_University/Year2/NLP/models/model_bow_l
/Users/snehanaidu/My_PC/City_University/Year2/NLP/models/model_tfidf
/Users/snehanaidu/My_PC/City_University/Year2/NLP/models/model_tfidf
/Users/snehanaidu/My_PC/City_University/Year2/NLP/models/model_tfidf
/Users/snehanaidu/My_PC/City_University/Year2/NLP/outputs/model_comp
/Users/snehanaidu/My_PC/City_University/Year2/NLP/outputs/cross_vali
```

Project complete.

Step 10: Test Set Evaluation (Pre-trained Models Only - No Retraining)

Use this section to evaluate pre-trained models without retraining.

This allows you to:

- Load the best two models (TF-IDF + NB and FinBERT) from the `models/` folder
- Evaluate them on the test set (2015-2016 data, 675 samples)
- Generate metrics WITHOUT any retraining

When to use this section:

- For coursework submission: Run ONLY this cell
- For reproducibility: Run the entire notebook OR just this cell

```
from sklearn.metrics import accuracy_score, precision_score, recall_s

# Load pre-trained models
with open(f'{MODELS_DIR}model_tfidf_nb.pkl', 'rb') as f:
    model_tfidf_nb_loaded, vectorizer_tfidf_nb = pickle.load(f)

with open(f'{MODELS_DIR}model_bow_lr.pkl', 'rb') as f:
    model_bow_lr_loaded, vectorizer_bow = pickle.load(f)

with open(f'{MODELS_DIR}model_tfidf_lr.pkl', 'rb') as f:
    model_tfidf_lr_loaded, vectorizer_tfidf_lr = pickle.load(f)

with open(f'{MODELS_DIR}model_tfidf_svm.pkl', 'rb') as f:
    model_tfidf_svm_loaded, vectorizer_tfidf_svm_test = pickle.load(f)

# Vectorize test set
X_test_bow_loaded = vectorizer_bow.transform(X_test)
X_test_tfidf_nb_loaded = vectorizer_tfidf_nb.transform(X_test)
X_test_tfidf_lr_loaded = vectorizer_tfidf_lr.transform(X_test)
X_test_tfidf_svm_loaded = vectorizer_tfidf_svm_test.transform(X_test)

# Predictions
y_pred_bow_test = model_bow_lr_loaded.predict(X_test_bow_loaded)
y_pred_tfidf_nb_test = model_tfidf_nb_loaded.predict(X_test_tfidf_nb_loaded)
y_pred_tfidf_lr_test = model_tfidf_lr_loaded.predict(X_test_tfidf_lr_loaded)
y_pred_tfidf_svm_test = model_tfidf_svm_loaded.predict(X_test_tfidf_svm_loaded)

# Evaluate all models (including FinBERT if available from training)
results_test = {
    'BoW + LogReg': {
        'accuracy': accuracy_score(y_test, y_pred_bow_test),
        'precision': precision_score(y_test, y_pred_bow_test),
```

```

        'recall': recall_score(y_test, y_pred_bow_test),
        'f1': f1_score(y_test, y_pred_bow_test)
    },
    'TF-IDF + LogReg': {
        'accuracy': accuracy_score(y_test, y_pred_tfidf_lr_test),
        'precision': precision_score(y_test, y_pred_tfidf_lr_test),
        'recall': recall_score(y_test, y_pred_tfidf_lr_test),
        'f1': f1_score(y_test, y_pred_tfidf_lr_test)
    },
    'TF-IDF + SVM': {
        'accuracy': accuracy_score(y_test, y_pred_tfidf_svm_test),
        'precision': precision_score(y_test, y_pred_tfidf_svm_test),
        'recall': recall_score(y_test, y_pred_tfidf_svm_test),
        'f1': f1_score(y_test, y_pred_tfidf_svm_test)
    },
    'TF-IDF + NB': {
        'accuracy': accuracy_score(y_test, y_pred_tfidf_nb_test),
        'precision': precision_score(y_test, y_pred_tfidf_nb_test),
        'recall': recall_score(y_test, y_pred_tfidf_nb_test),
        'f1': f1_score(y_test, y_pred_tfidf_nb_test)
    }
}

# Add FinBERT if predictions exist
if 'y_pred_bert' in locals():
    results_test['FinBERT'] = {
        'accuracy': accuracy_score(y_test, y_pred_bert),
        'precision': precision_score(y_test, y_pred_bert),
        'recall': recall_score(y_test, y_pred_bert),
        'f1': f1_score(y_test, y_pred_bert)
    }

# Display results
results_df = pd.DataFrame(results_test).T
print("\nTEST SET EVALUATION RESULTS")
print(results_df.round(4))

# Save test set results
results_df.to_csv(f'{OUTPUTS_DIR}test_set_evaluation_summary.csv')
print(f"\n✓ Saved to {OUTPUTS_DIR}test_set_evaluation_summary.csv")

```

TEST SET EVALUATION RESULTS

	accuracy	precision	recall	f1
BoW + LogReg	0.4683	0.5224	0.5000	0.5109
TF-IDF + LogReg	0.4921	0.5455	0.5143	0.5294
TF-IDF + SVM	0.5238	0.5833	0.5000	0.5385
TF-IDF + NB	0.5159	0.5405	0.8571	0.6630

✓ Saved to outputs/test_set_evaluation_summary.csv

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.